

1 Introdução

[TODO]

1.1 Objetivos

[TODO]

1.1.1 Objetivo Geral

[TODO]

1.1.2 Objetivos Específicos

[TODO]

1.2 Organização da monografia

[TODO]

Este trabalho é composto por XX capítulos. Neste Capítulo 1, proporcionamos uma introdução, justificativa e delineamos os objetivos...

2 Fundamentação Teórica

Este capítulo apresenta os conceitos necessários para a compreensão do trabalho. Inicialmente, são introduzidos fundamentos de aprendizado profundo e redes neurais artificiais, seguidos pelo processo de treinamento dessas redes. Em seguida, discute-se o problema do consumo energético em modelos de *Deep Learning* e, por fim, são apresentadas as técnicas de compressão de modelos, com destaque para a poda neural (*pruning*), que é o foco deste trabalho.

A exposição é deliberadamente progressiva e não pressupõe familiaridade prévia com aprendizado de máquina. Cada conceito é introduzido primeiro por meio de uma analogia com situações cotidianas e só então formalizado, de modo que o leitor possa acompanhar o raciocínio antes de encontrar a notação matemática que o descreve.

2.1 Aprendizado Profundo e Redes Neurais

2.1.1 Aprendizado Profundo (Deep Learning)

O Aprendizado Profundo é um campo especializado de aprendizado de máquina (GOODFELLOW; BENGIO; COURVILLE, 2016), caracterizado pelo uso de redes neurais artificiais com múltiplas camadas, o que permite aprender e extrair informações complexas a partir de grandes volumes de dados.

2.1.2 Redes Neurais Artificiais

A rede neural artificial é um modelo computacional inspirado na estrutura do cérebro humano, que busca imitar a forma como neurônios se conectam e processam sinais para aprender a partir de dados, identificando padrões e tomando decisões. Um exemplo clássico é o perceptron multicamadas (MLP), um modelo composto por funções matemáticas que mapeiam valores de entrada em valores de saída (GOODFELLOW; BENGIO; COURVILLE, 2016). Em tarefas de visão computacional, por exemplo, milhões de imagens de um objeto podem ser processadas para que a rede aprenda suas características visuais e seja capaz de reconhecê-lo posteriormente.

Uma analogia ajuda a fixar a ideia antes da definição formal. Imagine uma comissão encarregada de decidir se um pedido de empréstimo deve ser aprovado. Nenhum membro decide sozinho: cada um examina um aspecto diferente — a renda declarada, o histórico de pagamentos, o tempo de emprego — e emite um parecer. Esses pareceres não têm todos o mesmo valor, pois a comissão confia mais em uns avaliadores do que

em outros, e a decisão final combina todos eles, ponderados por essa confiança. Em uma rede neural, cada nó é um desses avaliadores e o peso de cada conexão é exatamente a confiança depositada naquele parecer. A diferença essencial é que a rede não recebe essas confianças prontas: ela as descobre sozinha, por tentativa e erro, ao longo do treinamento.

Uma rede neural profunda é composta por unidades (nós) interconectadas, organizadas em camadas.

- **Nó (neurônio):** É uma unidade computacional que calcula uma soma ponderada das entradas mais um *bias* e passa o resultado através de uma função de ativação, produzindo uma saída.
- **Camada de Entrada (Input Layer):** É onde o processo é iniciado e os dados são introduzidos na rede. O número de nós nessa camada corresponde ao número de características dos dados de entrada.
- **Camadas Ocultas (Hidden Layers):** São camadas intermediárias entre a entrada e a saída. Cada nó de uma camada oculta está conectado a todos os nós das camadas anterior e seguinte. É nelas que ocorre a maior parte do processamento e do aprendizado; a saída de uma camada oculta é utilizada como entrada da próxima, e, a cada camada, a rede passa a detectar características mais complexas e abstratas. Redes com múltiplas camadas ocultas são as chamadas redes de *Deep Learning*.
- **Camada de Saída (Output Layer):** Camada final da rede, onde o resultado é produzido. A estrutura da camada de saída depende da tarefa: em regressão, há um único nó com função de ativação linear; em classificação binária, um único nó expressa resultados do tipo sim/não; em classificação multiclasse, há um nó para cada classe possível, e o de maior ativação corresponde à predição final.

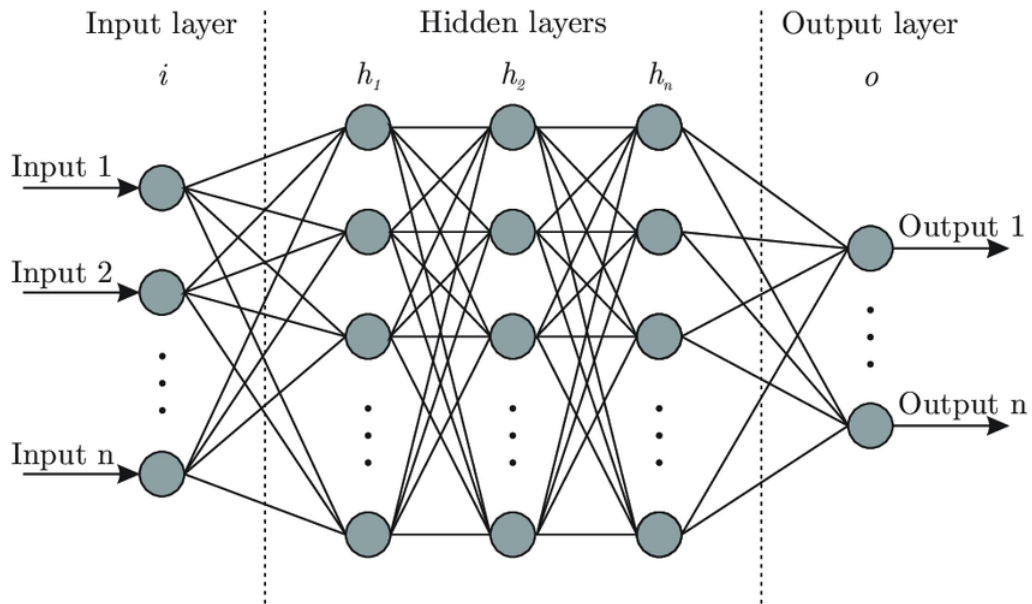
Cada conexão entre nós possui um peso associado, que determina a sua importância relativa. Pesos altos amplificam o sinal de entrada, enquanto pesos próximos de zero tornam a conexão pouco influente. Os pesos são inicializados com valores aleatórios e ajustados durante o treinamento.

2.1.3 Bias em Redes Neurais

O *bias* é um parâmetro adicional que ajusta a saída de um neurônio. Trata-se de um valor numérico associado a cada neurônio, não ligado a nenhuma entrada específica, atuando como um coeficiente linear. Antes da aplicação da função de ativação, o *bias* é somado ao resultado da soma ponderada das entradas.

A principal função do *bias* é deslocar a função de ativação, permitindo que o modelo se ajuste melhor aos dados. Sem ele, a soma ponderada precisaria ser alta para

Figura 1 – Arquitetura típica de uma rede neural artificial com uma camada de entrada, duas camadas ocultas e uma camada de saída.



que o neurônio fosse ativado; com o *bias*, um neurônio pode ser ativado mesmo com uma soma ponderada baixa, caso seu *bias* seja suficientemente alto. Inversamente, um *bias* muito negativo dificulta a ativação do neurônio. O *bias* é um parâmetro aprendível: inicialmente é inicializado com zero ou valores próximos de zero e, durante o treinamento, é atualizado pelo mesmo processo de retropropagação e descida de gradiente utilizado para os pesos.

Retomando a comissão de crédito: se os pesos representam o quanto se confia em cada avaliador, o *bias* representa o quanto o avaliador é exigente. Um membro benevolente aprova o pedido mesmo diante de evidências modestas; um membro rigoroso só aprova quando as evidências são fortes. São duas coisas independentes — confiar muito em alguém não diz nada sobre o quanto essa pessoa é exigente — e é por isso que a rede precisa aprender os dois conjuntos de valores separadamente.

2.1.4 Função de Ativação

A função de ativação é uma função matemática aplicada ao resultado da soma ponderada das entradas de um neurônio (já acrescido do *bias*), determinando a saída do neurônio. Sua principal função é introduzir não linearidade no modelo: sem ela, a composição de diversas camadas resultaria em uma transformação puramente linear, equivalente a uma rede com uma única camada, o que limitaria drasticamente a capacidade expressiva do modelo (GOODFELLOW; BENGIO; COURVILLE, 2016).

A razão de existir dessa função merece um exemplo concreto, pois é ela que justifica o próprio empilhamento de camadas. Aplicar dois descontos sucessivos em um preço

— um de 10% e outro de 20% — produz exatamente o mesmo efeito que aplicar um único desconto de 28%: encadear operações lineares nunca gera nada além de outra operação linear. Com redes neurais acontece o mesmo. Sem a função de ativação, uma rede de cem camadas calcularia exatamente aquilo que uma única camada já calcularia, e toda a profundidade seria desperdiçada. A não linearidade é o que quebra esse encadeamento e permite que cada camada acrescente algo que a anterior não conseguia representar.

A ativação de um neurônio ocorre em três etapas:

- Soma ponderada: o neurônio multiplica o valor de ativação de cada nó da camada anterior pelo peso da respectiva conexão e soma todos os resultados.
- Adição do *bias*: ao resultado da soma ponderada é somado o valor do *bias*, que auxilia o neurônio a ativar-se mais ou menos facilmente.
- Função de ativação: o resultado final (soma ponderada acrescida do *bias*) é passado pela função de ativação, cuja saída é então transmitida aos nós da próxima camada.

Figura 2 – Representação matricial da propagação de um neurônio, da soma ponderada à função de ativação (Sigmoide).

$$\underbrace{\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix}}_{\text{ativações de saída}} = \sigma \left(\underbrace{\begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix}}_{\text{pesos } W} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}_{\text{entradas da camada anterior}} + \underbrace{\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}}_{\text{bias}} \right)$$

$\sigma(z)$

a função de ativação comprime qualquer valor de z no intervalo $(0, 1)$

Fonte: Adaptado de Nielsen (2015).

Há diversas funções de ativação, cada uma com suas próprias características e aplicações.

- **ReLU (Rectified Linear Unit):** Função de ativação mais utilizada em redes neurais modernas, proposta para redes profundas por Nair e Hinton (2010) e popularizada por Krizhevsky, Sutskever e Hinton (2012). É uma função linear por partes, com inclinação 1 para valores positivos e 0 para valores negativos. É computacionalmente muito eficiente e não sofre do problema do desaparecimento do gradiente

para entradas positivas. Em contrapartida, pode sofrer do problema conhecido como *Dying ReLU*, no qual neurônios cujas entradas se tornam persistentemente negativas deixam de ser ativados e param de aprender.

- **Sigmoide:** Função que mapeia qualquer valor real para o intervalo $(0, 1)$, sendo útil em classificação binária, já que a saída pode ser interpretada como uma probabilidade. Caiu em desuso em camadas ocultas porque, para entradas com magnitude muito alta ou muito baixa, sua derivada se aproxima de zero, fenômeno conhecido como desaparecimento do gradiente.
- **Tanh (Tangente Hiperbólica):** Semelhante à Sigmoide, porém mapeando os valores para o intervalo $(-1, 1)$, o que a torna centrada em zero e costuma favorecer a convergência. Também sofre do problema do desaparecimento do gradiente.
- **Softmax:** Diferentemente das demais, não é aplicada a neurônios individuais de camadas ocultas, mas sim ao conjunto dos neurônios da camada de saída em tarefas de classificação multiclasse. Transforma as saídas brutas da última camada em uma distribuição de probabilidades, com valores no intervalo $(0, 1)$ que somam 1. Na prática, converte pontuações sem escala definida em porcentagens comparáveis: o modelo deixa de responder “7,2 para gato e 3,1 para cachorro” e passa a responder “85% de chance de ser gato, 15% de ser cachorro”.

Em cada camada, as ativações produzidas determinam as ativações da camada seguinte. De camada em camada, esses padrões tornam-se progressivamente mais complexos e abstratos, até que a camada de saída produz o resultado final. Assim, camadas iniciais aprendem características simples, enquanto camadas mais profundas combinam essas características para representar conceitos de alto nível.

2.2 Treinamento de Redes Neurais

2.2.1 Gradiente

No contexto de redes neurais, o gradiente corresponde ao vetor de derivadas parciais da função de perda em relação aos pesos da rede. Ele é o elemento central que permite a minimização da função de perda por meio do algoritmo da descida do gradiente, um método de primeira ordem que utiliza as informações do gradiente para atualizar os parâmetros da rede (GOODFELLOW; BENGIO; COURVILLE, 2016).

Cada componente do vetor de gradiente é uma derivada parcial, calculada assumindo que as demais variáveis são mantidas constantes. A magnitude da derivada indica o quanto a função está variando; o seu sinal, a direção dessa variação: positivo, a função está aumentando; negativo, diminuindo.

Uma analogia comum é a de um caminhante que desce uma encosta em meio à neblina, enxergando apenas o solo ao seu redor: a cada passo, ele sente para qual lado o terreno mais desce (o oposto do gradiente) e avança nessa direção. Na descida do gradiente, a “encosta” é a função de perda e os “passos” são os ajustes nos pesos, repetidos até alcançar um ponto de mínimo.

2.2.2 Backpropagation

A retropropagação (*backpropagation*) é o algoritmo que permite o aprendizado das redes neurais, por possibilitar o cálculo da contribuição de cada parâmetro (peso ou *bias*) para o erro da predição (RUMELHART; HINTON; WILLIAMS, 1986). Uma vez que o erro é calculado na camada de saída, o algoritmo executa um fluxo invertido: da saída em direção à entrada, propagando o erro para as camadas anteriores.

A lógica é a mesma de uma apuração de responsabilidades em uma linha de produção. Sabe-se apenas que o produto final saiu defeituoso; para corrigir o processo, é preciso percorrer a linha de trás para frente e estimar o quanto cada etapa contribuiu para o defeito. Uma etapa que respondeu por boa parte do problema precisa de um ajuste grande; outra que quase não influiu, de um ajuste pequeno. O *backpropagation* faz esse rastreamento de forma matemática, atribuindo a cada peso da rede uma parcela de responsabilidade pelo erro.

Para saber como cada peso individual se correlaciona com o erro, o algoritmo calcula a derivada parcial da função de perda em relação a esse peso, utilizando a regra da cadeia multivariada para decompor o cálculo em derivadas intermediárias. Com todas as derivadas obtidas, um algoritmo de otimização atualiza cada peso na direção oposta ao gradiente:

$$w_{novo} = w_{antigo} - \eta \cdot \frac{\partial L}{\partial w}$$

em que η é a taxa de aprendizado, hiperparâmetro que controla o tamanho do ajuste aplicado a cada iteração.

Voltando ao caminhante na neblina, a taxa de aprendizado é o tamanho do passo que ele dá. Passos curtos demais tornam a descida segura, porém lenta a ponto de o treinamento não terminar em tempo viável. Passos longos demais fazem o caminhante saltar por cima do fundo do vale e subir a encosta oposta, oscilando indefinidamente sem nunca chegar ao ponto mais baixo. Encontrar um valor adequado é uma das decisões mais sensíveis do treinamento.

2.2.3 Hiperparâmetros

Hiperparâmetros são variáveis externas que definem tanto o algoritmo de aprendizado quanto a arquitetura da rede neural. Diferentemente dos parâmetros do modelo (pesos e *biases*), que são ajustados automaticamente pelo *backpropagation*, os hiperparâmetros são definidos antes do início do treinamento e controlam como a rede aprende a partir dos dados. Existem técnicas automatizadas para sua busca, como *grid search* e otimização bayesiana, mas o ajuste manual baseado em experiência ainda é amplamente empregado.

A distinção entre parâmetros e hiperparâmetros é a mesma que separa, em uma cozinha, as escolhas do cozinheiro do resultado do cozimento. A temperatura do forno e o tempo de preparo são definidos antes de o prato ir ao fogo e não se alteram sozinhos: são os hiperparâmetros. O ponto da massa e a cor da crosta, por outro lado, são produzidos pelo processo: são os parâmetros. Errar a temperatura não impede o prato de assar, mas compromete o resultado — e, do mesmo modo, hiperparâmetros mal escolhidos não impedem o treinamento de ocorrer, apenas fazem com que ele chegue a um modelo pior.

Em geral, os hiperparâmetros podem ser agrupados em duas categorias:

- **Hiperparâmetros de arquitetura:** definem a estrutura da rede, como o número de camadas ocultas, o número de neurônios em cada camada e o tipo de função de ativação utilizada.
- **Hiperparâmetros de treinamento:** controlam como o modelo se ajusta aos dados, como a taxa de aprendizado, o número de épocas, o tamanho do lote (*batch size*) e a escolha do algoritmo de otimização.

2.2.4 Passo a passo do treinamento

O treinamento de uma rede neural supervisionada requer um grande conjunto de dados rotulados, isto é, cada exemplo de entrada deve estar associado a uma saída correta. Esse conjunto é tradicionalmente dividido em três subconjuntos (GOODFELLOW; BENGIO; COURVILLE, 2016):

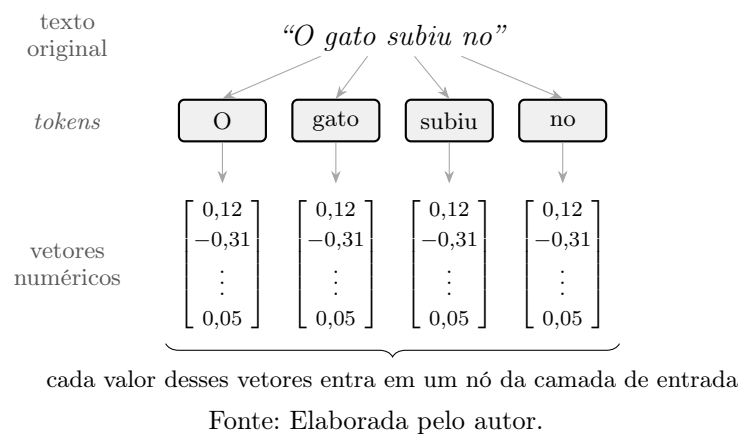
- **Conjunto de treinamento**, usado para ajustar os pesos e *biases*;
- **Conjunto de validação**, usado para monitorar o desempenho durante o treinamento e ajustar hiperparâmetros;
- **Conjunto de teste**, usado para avaliar o desempenho final do modelo de forma imparcial.

Além disso, é necessário definir a arquitetura da rede (camadas, neurônios, funções de ativação) e os hiperparâmetros de treinamento. O treinamento ocorre ao longo de diversas iterações, nas quais cada passagem completa pelo conjunto de treinamento é chamada de época (*epoch*); em cada época, os dados são processados em pequenos lotes (*batches*).

A analogia mais direta é a de um estudante diante de um livro. Uma época equivale a percorrer o livro inteiro uma vez; os lotes são os trechos em que essa leitura é dividida, com uma pausa para revisão ao fim de cada um. E, assim como raramente basta uma única leitura para fixar o conteúdo, a rede precisa percorrer o conjunto de treinamento diversas vezes até que os ajustes se estabilizem.

Como exemplo, considere um modelo de linguagem que recebe a frase “O gato subiu no”. Antes de qualquer cálculo, o texto precisa virar número: ele é fragmentado em unidades menores, chamadas *tokens*, e cada *token* é convertido em um vetor de valores numéricos. São esses vetores que alimentam a camada de entrada — cada posição do vetor corresponde a um nó, exatamente como, em uma rede que classifica imagens de 28×28 pixels, cada um dos 784 nós de entrada receberia a intensidade de um pixel. Em ambos os casos vale a mesma regra: um nó de entrada para cada característica do dado.

Figura 3 – Da frase aos nós da camada de entrada: o texto é fragmentado em *tokens* e cada *token* é convertido em um vetor numérico.



O ciclo de treinamento pode ser resumido nos seguintes passos:

- **Passo 1 – Inicialização dos parâmetros:** pesos são inicializados com valores aleatórios baixos e *biases* com zero. Essa quebra de simetria permite que diferentes neurônios aprendam características distintas.
- **Passo 2 – *Forward pass*:** um lote de dados é inserido na camada de entrada e propagado através das camadas ocultas até a camada de saída, produzindo uma predição.

- **Passo 3 – Cálculo da perda:** a predição é comparada com o rótulo verdadeiro por meio de uma função de perda, que quantifica numericamente o quão distante a predição está do valor correto.
- **Passo 4 – *Backpropagation*:** o erro é propagado de volta pela rede, calculando-se o gradiente da função de perda em relação a cada peso e *bias*.
- **Passo 5 – Atualização dos parâmetros:** um otimizador atualiza pesos e *biases* na direção oposta ao gradiente, reduzindo a função de perda.
- **Passo 6 – Repetição:** os passos 2 a 5 são repetidos para cada lote e ao longo de um número definido de épocas.

O treinamento é interrompido quando o modelo atinge uma precisão satisfatória ou quando a função de perda deixa de diminuir significativamente.

2.3 Arquitetura Transformer

A arquitetura *Transformer*, proposta por Vaswani et al. (2017) no trabalho *Attention is All You Need*, marcou uma mudança profunda na forma como modelos de aprendizado profundo processam sequências de texto. Antes dessa proposta, o campo era dominado por arquiteturas que processavam a sequência elemento por elemento, palavra por palavra. O *Transformer* rompeu com essa abordagem ao introduzir um mecanismo capaz de considerar toda a sequência de uma só vez. Esta seção apresenta, de forma progressiva, os conceitos necessários para compreender a arquitetura utilizada neste trabalho: as limitações das abordagens anteriores, o mecanismo de atenção, a operação de autoatenção e sua versão multi-cabeça, a estrutura de cada bloco da rede e, finalmente, o modelo GPT-2, que é o objeto central dos experimentos de poda.

2.3.1 O Problema que o Transformer Resolve

Toda arquitetura nasce como resposta a uma limitação da anterior, e o *Transformer* não é exceção. Compreender qual era essa limitação é o que transforma o mecanismo de atenção — apresentado a seguir — em uma solução, e não em uma escolha arbitrária.

Até 2017, o processamento de texto era dominado pelas redes neurais recorrentes (*Recurrent Neural Networks*, RNNs). Elas leem a frase palavra por palavra, na ordem em que aparece, carregando consigo um resumo do que já foi lido: a cada nova palavra, esse resumo é atualizado. É, em essência, o modo como uma pessoa lê um texto em voz alta, guardando na memória o que veio antes. Dessa mecânica decorrem os dois problemas que motivaram o *Transformer*.

O primeiro é o **esquecimento**. Considere o trecho “A Inglaterra é minha terra natal. Passei toda a minha vida lá. Mudei-me para a Espanha há dois dias. Eu falo apenas uma língua, que é o ____”. Preencher a lacuna com “inglês” exige recuperar uma palavra que apareceu muito antes. Como o resumo é reescrito a cada passo, a informação antiga vai se diluindo, e a influência de “Inglaterra” chega enfraquecida ao final. Tecnicamente, o fenômeno se manifesta durante o treinamento como gradiente desaparecente (*vanishing gradient*): os gradientes são multiplicados repetidamente pelas matrizes de peso e, quando esses valores são pequenos, encolhem até se aproximarem de zero (BENGIO; SIMARD; FRASCONI, 1994). É a limitação conhecida como incapacidade de lidar com **dependências de longo alcance**. Arquiteturas posteriores, como as redes LSTM (*Long Short-Term Memory*), atenuaram o esquecimento por meio de mecanismos que regulam o que deve ser mantido ou descartado (HOCHREITER; SCHMIDHUBER, 1997), mas não resolveram o segundo problema.

O segundo é a **lentidão**, e é o que mais interessa a este trabalho. Como cada palavra só pode ser processada depois da anterior, a leitura é obrigatoriamente sequencial: não há como dividir o trabalho entre vários processadores. Isso desperdiça justamente a principal característica das unidades de processamento gráfico (GPUs) modernas, que é executar milhares de operações ao mesmo tempo. A analogia é a de um supermercado com vinte caixas abertos, mas com uma regra que obrigue os clientes a passar por um de cada vez: os outros dezenove ficam ociosos, e a fila anda no ritmo de um só.

Foi para atacar essas duas limitações simultaneamente que se desenvolveu o mecanismo de atenção.

2.3.2 Mecanismo de Atenção

O passo seguinte dessa evolução foi o mecanismo de atenção (*attention mechanism*), introduzido por Bahdanau, Cho e Bengio (2015) no contexto da tradução automática. Em vez de comprimir toda a frase em um único estado oculto, a atenção permite que o modelo “olhe” de volta e foque seletivamente em partes específicas da entrada. No exemplo anterior, ela pode consultar diretamente o termo “Inglaterra”, independentemente de quão distante ele esteja da lacuna, eliminando o risco de o gradiente desaparecer pelo caminho.

Para operacionalizar essa ideia, o mecanismo utiliza três conceitos: a consulta (*query*), a chave (*key*) e o valor (*value*). Uma analogia útil é a de uma biblioteca: cada livro tem uma etiqueta na lombada (a *chave*) que descreve seu conteúdo; quando um leitor chega com uma pergunta (a *consulta*), ela é comparada com cada etiqueta para decidir quais livros são mais relevantes; o conteúdo desses livros (os *valores*) é então combinado, com os mais relevantes contribuindo mais para a resposta. No *Transformer*, esse processo ocorre com números: a relevância entre uma consulta e cada chave é calculada

matematicamente, e os valores são somados com pesos proporcionais a essa relevância.

A inovação central de Vaswani et al. (2017), em 2017, foi construir uma arquitetura inteira baseada exclusivamente nesse mecanismo, dispensando por completo a recorrência. Com isso, todas as posições de uma sequência se comunicam diretamente em uma única operação, sem precisar “passar” por palavras intermediárias. Isso resolve as duas limitações das RNNs de uma só vez: as dependências de longo alcance tornam-se tão fáceis de capturar quanto as de curto alcance, e o cálculo passa a ser paralelizável, tornando as arquiteturas baseadas em RNN e LSTM obsoletas para muitas tarefas modernas.

2.3.3 Autoatenção e Atenção Multi-Cabeça

A variante do mecanismo de atenção utilizada no *Transformer* é chamada de autoatenção (*self-attention*). O prefixo “auto” indica que a consulta, a chave e o valor são todos derivados da mesma sequência de entrada. Ou seja, ao processar uma frase, cada palavra pergunta a si mesma: “quais outras palavras desta frase são relevantes para minha interpretação?”. Exemplo: “O gato que estava no telhado dormiu”. Ao processar a palavra “dormiu”, o mecanismo de autoatenção calcula um peso para cada palavra da frase. A palavra “gato” recebe peso alto, pois é o sujeito do verbo; palavras como “telhado” ou “no” recebem pesos baixos, pois são menos relevantes para determinar quem dormiu.

Na prática, esses pesos são calculados da seguinte forma, a partir de cada representação de entrada, três matrizes de pesos aprendíveis durante o treinamento transformam cada representação em três vetores: o vetor de consulta Q (*query*), o vetor de chave K (*key*) e o vetor de valor V (*value*). O grau de compatibilidade entre uma consulta e cada chave é calculado pelo produto escalar entre eles. Esses valores brutos de compatibilidade passam pela função *softmax*, que os converte em uma distribuição de probabilidades, isto é, um conjunto de números entre 0 e 1 que somam exatamente 1. Esses pesos normalizados determinam com que intensidade cada valor V contribui para o resultado final, que é obtido pela soma ponderada dos valores.

Essa operação completa é denominada *Scaled Dot-Product Attention* (atenção por produto escalar escalado) e é definida formalmente por (VASWANI et al., 2017):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Em termos intuitivos, o produto $QK^\top$ mede a compatibilidade entre todas as consultas e todas as chaves de uma só vez; a divisão por $\sqrt{d_k}$ (a dimensão das chaves) é um fator de escala que estabiliza o treinamento; a *softmax* converte essas pontuações em pesos que somam 1; e a multiplicação por V produz, para cada posição, uma média ponderada dos valores, isto é, sua representação já contextualizada pelo restante da sequência.

Uma única operação de atenção, porém, captura apenas um tipo de relação entre as palavras de cada vez. Para superar essa limitação, Vaswani et al. (2017) propõem a atenção multi-cabeça (*multi-head attention*). A analogia é a seguinte: se a atenção simples é como um único leitor relendo a frase em busca de relações gramaticais, a atenção multi-cabeça é como h leitores independentes, cada um procurando um tipo diferente de relação. Um leitor pode focar em concordância verbal (“gato”/“dormiu”), outro em relações de posse (“telhado do vizinho”), outro em referências pronominais, e assim por diante. O GPT-2 *small*, por exemplo, utiliza 12 cabeças de atenção por bloco.

Na prática, cada uma das h cabeças executa a operação *Scaled Dot-Product Attention* de forma independente, com suas próprias matrizes de projeção aprendíveis para Q , K e V . As saídas das h cabeças são depois concatenadas e projetadas por uma matriz linear aprendível, produzindo a representação final de cada posição. Desse modo, o modelo pode simultaneamente capturar relações em diferentes subespaços semânticos e sintáticos, sem custo adicional expressivo em relação à atenção simples.

Sob a perspectiva da compressão de modelos, as cabeças de atenção constituem unidades podáveis naturais para a poda estruturada: estudos da literatura demonstram que nem todas as cabeças são igualmente úteis, e que algumas podem ser removidas inteiramente sem degradação significativa da qualidade das previsões (MICHEL; LEVY; NEUBIG, 2019).

2.3.4 Estrutura do Bloco Transformer

Para entender como o *Transformer* processa informação em profundidade, é útil pensar em cada bloco como uma “estação de trabalho” em uma linha de montagem. Na primeira bancada, as palavras trocam informações entre si por meio da atenção multi-cabeça: cada palavra atualiza sua representação consultando todas as demais. Na segunda bancada, cada palavra refina individualmente sua própria representação, de forma independente das outras, por meio de uma pequena rede totalmente conectada. Esse par de operações se repete em cada bloco, e os blocos são empilhados, fazendo com que as representações das palavras se tornem progressivamente mais ricas e abstratas a cada andar.

O primeiro bloco é a camada de atenção multi-cabeça, já apresentada na seção anterior, responsável por integrar informação contextual de toda a sequência. O segundo é uma rede *feedforward* totalmente conectada — um MLP (*multilayer perceptron*) aplicado de forma idêntica e independente a cada posição. Esse MLP é composto por duas transformações lineares com uma função de ativação não linear entre elas: a função GELU (*Gaussian Error Linear Unit*) (HENDRYCKS; GIMPEL, 2016) nos modelos GPT, ou ReLU em outras variantes. Diferentemente da ReLU (NAIR; HINTON, 2010), que simplesmente zera valores negativos, a GELU faz uma transição suave em torno da origem.

Na prática, essa suavidade evita o problema dos “neurônios mortos” (*dying ReLU*), em que unidades deixam de aprender por receberem gradiente nulo, e por isso Radford et al. (2019) a adotaram como ativação padrão na família GPT. A dimensão intermediária desse MLP é tipicamente quatro vezes maior do que a dimensão do modelo — no GPT-2 *small*, a dimensão do modelo é 768 e a dimensão intermediária é 3.072 — o que confere ao bloco grande capacidade de representação antes de comprimir novamente a informação.

2.3.5 Modelos de Linguagem de Grande Escala e o GPT-2

Modelos de linguagem de grande escala (*Large Language Models*, LLMs) são redes neurais profundas, tipicamente baseadas na arquitetura *Transformer*, treinadas em grandes volumes de texto com o objetivo de aprender a prever palavras. A tarefa de pré-treinamento é simples de enunciar: dado um trecho de texto, o modelo deve prever qual token vem a seguir. Um *token* é a unidade mínima de texto que o modelo processa — pode ser uma palavra, parte de uma palavra, um sinal de pontuação ou um espaço. Dada a sequência “O aluno abriu o”, um LLM bem treinado atribuiria alta probabilidade a tokens como “livro”, “caderno” ou “computador”, e baixa probabilidade a tokens como “foguetes” ou “plutônio”. Repetida bilhões de vezes sobre textos variados, essa tarefa aparentemente simples força o modelo a aprender gramática, fatos sobre o mundo, raciocínio básico e muito mais (RADFORD et al., 2019).

Para que o texto possa ser processado numericamente, ele precisa primeiro ser convertido em tokens e depois em vetores numéricos. Essa conversão ocorre em duas etapas. A primeira é a tokenização, realizada por algoritmos como o BPE (*Byte Pair Encoding*), que divide o texto em unidades de comprimento variável de forma a equilibrar cobertura e eficiência. Por exemplo, a palavra “inconstitucional” pode ser dividida em tokens como “in”, “constitu” e “cional”. O GPT-2 possui um vocabulário de aproximadamente 50.257 tokens. A segunda etapa converte cada token em um vetor numérico denso por meio de uma camada de *embedding*: o modelo aprende, durante o treinamento, uma representação vetorial para cada token do vocabulário, de modo que tokens semanticamente próximos tendam a ter representações próximas no espaço vetorial.

O GPT-2 (*Generative Pre-trained Transformer 2*), proposto por Radford et al. (2019) pela OpenAI em 2019, é um modelo autorregressivo baseado exclusivamente no componente decodificador do *Transformer* original. Ele foi pré-treinado em um corpus de aproximadamente 40 GB de texto coletado da internet, abrangendo uma grande diversidade de domínios e estilos. A variante GPT-2 *small*, utilizada neste trabalho, possui 12 blocos *Transformer* empilhados, 12 cabeças de atenção por bloco, dimensão de modelo igual a 768 e aproximadamente 124 milhões de parâmetros treináveis. Apesar de ser a menor variante da família GPT-2, esse número de parâmetros é cerca de cinco vezes maior do que o de uma ResNet-50 (25 milhões de parâmetros), arquitetura de referência

nos estudos clássicos de poda — e as variantes maiores da própria família GPT-2 chegam a 1,5 bilhão de parâmetros, o que ilustra a escala característica dos modelos baseados em *Transformer*.

O crescimento em escala dos LLMs ao longo dos anos seguintes foi vertiginoso: o GPT-3, publicado em 2020, possui 175 bilhões de parâmetros; o PaLM, do Google, chega a 540 bilhões. Mesmo modelos de médio porte como o GPT-2 impõem custos energéticos e computacionais consideráveis em ambientes de produção, onde a inferência é executada milhares ou milhões de vezes por dia. A poda estruturada surge, nesse contexto, como uma estratégia concreta para reduzir esses custos: a remoção de cabeças de atenção redundantes e de neurônios MLP de baixa importância pode diminuir substancialmente a contagem de operações por inferência, com impacto controlado sobre a qualidade das previsões (PATTERSON et al., 2021).

2.4 Consumo Energético em Redes Neurais

O avanço de técnicas e *hardware* voltados ao treinamento de redes neurais, motivado pela busca por maior precisão, trouxe consigo um consumo de energia considerável e, conseqüentemente, uma significativa emissão de dióxido de carbono. Strubell, Ganesh e McCallum (2019) relatam, por exemplo, que o treinamento de um modelo *transformer* com busca de arquitetura neural pode emitir cerca de 626.155 libras de CO_2 , enquanto a emissão média de um automóvel ao longo de toda a sua vida útil é de aproximadamente 126.000 libras — cerca de cinco vezes menos.

Esse custo energético concentra-se em duas etapas do ciclo de vida de um modelo: o treinamento e a inferência.

No **treinamento**, o modelo é alimentado com grandes volumes de dados e ajustado iterativamente pelo *backpropagation*. Esse processo é computacionalmente intenso e geralmente requer *hardware* especializado (GPUs ou TPUs), muitas vezes organizado em *clusters* com centenas ou milhares de aceleradores.

Na **inferência**, o modelo já treinado é utilizado para gerar previsões. O custo de uma única predição é relativamente baixo; contudo, em contextos de larga escala — como os modelos de IA generativa servindo milhões de requisições diárias — o custo energético agregado é expressivo (LUCCIONI; VIGUIER; LIGOZAT, 2022).

A diferença entre as duas etapas é de natureza, não apenas de grandeza. O treinamento é um gasto concentrado e único, comparável ao custo de construir uma ponte; a inferência é um gasto pequeno, porém perpétuo, comparável ao custo de mantê-la aberta ao tráfego. Uma ponte muito usada acaba custando, ao longo de décadas, mais em manutenção do que custou para ser erguida — e o mesmo ocorre com um modelo consultado

milhões de vezes por dia. É por essa razão que este trabalho concentra-se na eficiência da inferência.

2.5 Compressão de Modelos

Com o crescimento do *Deep Learning*, os modelos de IA vêm dobrando em tamanho a cada poucos anos (SEVILLA et al., 2022). Modelos de alta performance costumam ser otimizados exclusivamente para acurácia, resultando em redes com grande quantidade de parâmetros, que ocupam muito espaço de armazenamento e demandam um elevado número de operações em ponto flutuante (FLOPs) para serem executadas. Isso é particularmente problemático em dispositivos com capacidade limitada, como *smartphones* e sistemas embarcados, e também em contextos de uso em larga escala, nos quais o custo energético se torna significativo.

A compressão de modelos é um campo que reúne técnicas voltadas a atenuar esses problemas, reduzindo o tamanho dos modelos e/ou a quantidade de operações necessárias para sua execução, preservando, tanto quanto possível, a acurácia. As redes neurais modernas possuem alta redundância de parâmetros (DENIL et al., 2013), o que torna viável remover ou simplificar aqueles com baixa contribuição ao resultado final.

Essa redundância não é um defeito, mas uma consequência do modo como esses modelos são construídos. Como não se sabe de antemão quantos parâmetros a tarefa exige, projeta-se a rede com folga generosa, do mesmo modo que se dimensiona uma instalação elétrica para uma carga muito acima da que será efetivamente usada. A folga facilita o treinamento, mas permanece no modelo depois que ele já aprendeu — e é justamente essa sobra que as técnicas de compressão procuram eliminar.

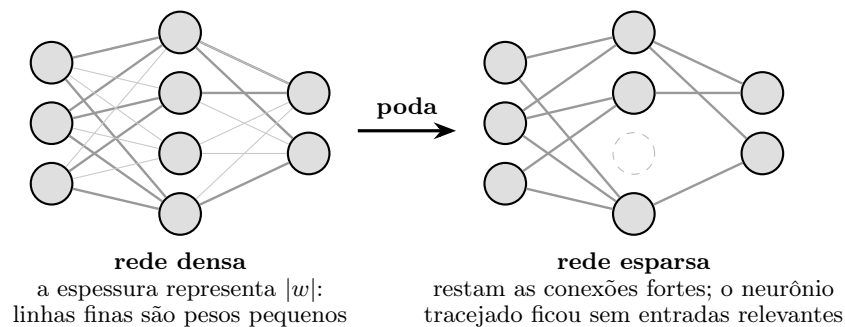
As quatro principais famílias de técnicas de compressão são:

- **Quantização (Quantization):** reduz o número de *bits* utilizados para representar cada peso do modelo. Em vez de pontos flutuantes de 32 bits, os parâmetros passam a ser representados com precisão reduzida (16 ou 8 bits, por exemplo), o que diminui o tamanho do modelo e pode acelerar operações em *hardware* que suporta aritmética de menor precisão (JACOB et al., 2018). O equivalente cotidiano é anotar preços arredondados: registrar R\$ 20 em vez de R\$ 19,99 ocupa menos espaço e agiliza as contas, e na maioria das situações a diferença é irrelevante.
- **Destilação de Conhecimento (Knowledge Distillation):** proposta por Hinton, Vinyals e Dean (2015), é uma metodologia de treinamento em que um modelo menor (*aluno*) aprende a partir de um modelo maior e mais preciso (*professor*). O aluno é treinado não apenas para acertar os rótulos verdadeiros, mas também para

imitar a distribuição de saída do professor, resultando em um modelo compacto que, frequentemente, alcança acurácia superior à que teria se fosse treinado do zero.

- **Fatoração de Matrizes (Matrix Factorization):** matrizes de pesos de grandes dimensões são decompostas em produto de matrizes menores cuja multiplicação aproxima a matriz original, reduzindo o custo computacional.
- **Poda Neural (Neural Pruning):** processo de remoção de pesos — ou de estruturas maiores, como filtros — que contribuem pouco para a saída do modelo. É a técnica central deste trabalho e será detalhada na Seção 2.6.

Figura 4 – Ilustração do processo de poda: a rede original (densa) tem conexões e/ou neurônios removidos, tornando-se esparsa.



Fonte: Adaptado de Han et al. (2015).

2.6 Poda Neural (Pruning)

A poda neural é uma técnica de compressão que visa reduzir o tamanho do modelo e o custo computacional da inferência por meio da remoção de parâmetros que pouco contribuem para o resultado final. O processo mais comum parte de um modelo previamente treinado, cujo foco inicial foi exclusivamente a acurácia, e aplica a poda como uma etapa posterior.

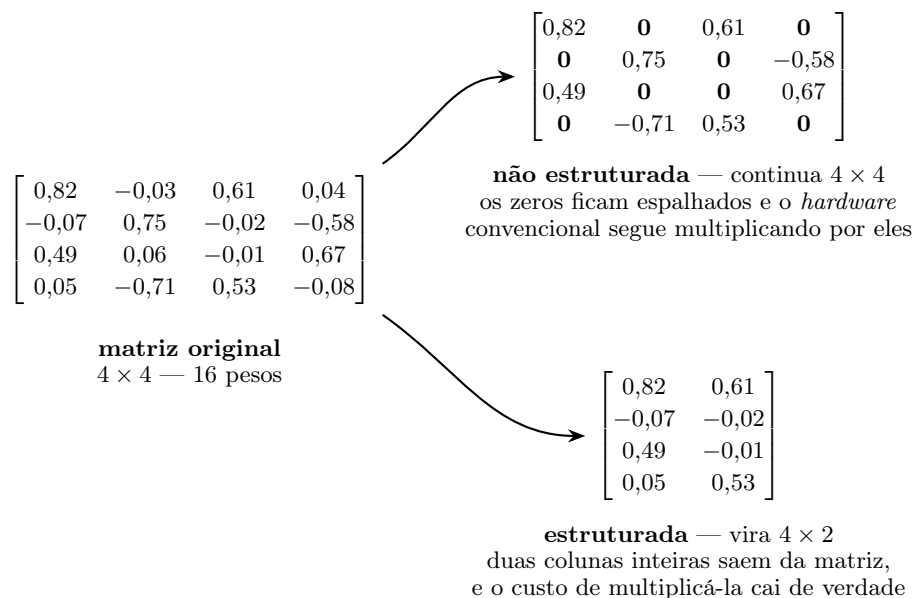
O termo é emprestado da jardinagem, e a metáfora é precisa: podar uma árvore consiste em cortar os galhos que consomem seiva sem produzir frutos, de modo que a planta fique mais leve e continue rendendo o mesmo. A dificuldade, tanto no jardim quanto na rede neural, está em identificar com segurança quais galhos são realmente dispensáveis — e é essa pergunta que separa as diferentes estratégias de poda apresentadas a seguir.

2.6.1 Tipos de Poda

Na literatura, duas categorias principais de poda são consideradas (BLALOCK et al., 2020):

- **Poda não estruturada:** remoção de pesos individuais, independentemente da sua posição na matriz de pesos. Esta abordagem oferece maior flexibilidade e, geralmente, taxas de compressão elevadas com perda reduzida de acurácia. Entretanto, do ponto de vista do custo de inferência, a poda não estruturada apresenta uma limitação importante: processadores e GPUs convencionais não são eficientes em pular multiplicações por zero espalhadas de forma aleatória na matriz de pesos. Assim, embora o modelo tenha muitos zeros, as operações ainda são executadas, e o ganho prático de velocidade e energia tende a ser marginal, a menos que se utilize *hardware* especializado ou formatos de armazenamento esparsos.
- **Poda estruturada:** remoção de estruturas inteiras, como neurônios, filtros ou canais convolucionais, o que implica a remoção completa de detectores de características. Como o resultado é uma matriz menor e ainda densa, há um ganho direto de velocidade e eficiência em *hardware* convencional. O custo dessa abordagem é uma potencial perda maior de acurácia para uma mesma taxa de compressão.

Figura 5 – As duas categorias de poda aplicadas à mesma matriz de pesos. Na poda não estruturada os pesos removidos são apenas zerados e a matriz conserva suas dimensões; na poda estruturada, colunas inteiras são eliminadas e a matriz encolhe de fato.



Fonte: Elaborada pelo autor.

A distinção entre as duas categorias é o que separa um ganho real de um ganho apenas contábil, e uma analogia com um livro a torna evidente. A poda não estruturada equivale a apagar palavras espalhadas por todas as páginas: o texto fica mais enxuto, mas o livro continua com o mesmo número de páginas, e quem for lê-lo ainda terá de folhear todas elas — inclusive as partes apagadas. A poda estruturada equivale a arrancar

capítulos inteiros: o livro encolhe de fato, pesa menos e é percorrido em menos tempo. Como o *hardware* convencional não sabe pular um espaço em branco no meio de uma multiplicação, apenas a segunda forma se converte em redução efetiva de tempo e de energia. Essa diferença é retomada no Capítulo 6, onde é verificada experimentalmente.

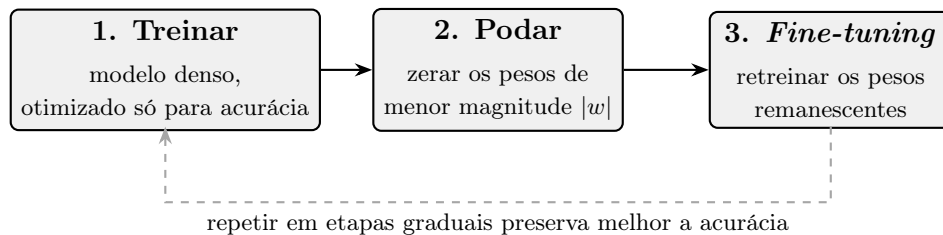
Cabe mencionar também a chamada **esparsidade estruturada** (por exemplo, padrões 2:4), em que o algoritmo é forçado a zerar, a cada bloco de quatro pesos, exatamente dois. Como o *hardware* sabe previamente quantos e em que formato os zeros aparecem, a matriz pode ser comprimida de maneira determinística, permitindo acelerações reais em GPUs modernas que suportam esse padrão (NVIDIA Corporation, 2020).

Assim, a poda estruturada é a que apresenta benefícios mais imediatos em termos de FLOPs e consumo de energia em *hardware* convencional, razão pela qual ela recebe destaque neste trabalho.

2.6.2 Poda por Magnitude

A técnica mais simples e amplamente estudada é a poda por magnitude. Nela, considera-se que pesos cujos valores absolutos são próximos de zero contribuem pouco para a saída do modelo e, portanto, podem ser removidos. Define-se um limiar (ou uma taxa de esparsidade desejada) e todos os pesos abaixo desse limiar são zerados, criando redes esparsas (HAN et al., 2015).

Figura 6 – Pipeline clássico de poda por magnitude: treinamento do modelo denso, remoção dos pesos de menor magnitude e retreinamento (*fine-tuning*) dos pesos remanescentes.



Fonte: Adaptado de Han et al. (2015).

2.6.3 Pipeline Geral da Poda

De forma geral, a poda pode ser dividida nos seguintes passos:

- **Passo 1 – Definição de critério e taxa de poda:** partindo de um modelo já treinado, define-se o critério de importância dos parâmetros (magnitude absoluta, norma L_1 de filtros, sensibilidade, entre outros) e a taxa de esparsidade desejada.

- **Passo 2 – Identificação dos parâmetros a podar:** com base no critério escolhido, o algoritmo classifica os parâmetros por importância e seleciona os menos relevantes.
- **Passo 3 – Execução da poda:** os parâmetros selecionados são zerados (no caso da poda não estruturada) ou fisicamente removidos (no caso da poda estruturada), gerando um modelo esparso ou uma nova arquitetura reduzida.
- **Passo 4 – Ajuste fino (*fine-tuning*):** como a remoção tende a deteriorar a acurácia, o modelo podado é retreinado com uma taxa de aprendizado reduzida, para que os parâmetros remanescentes compensem a ausência dos que foram removidos.
- **Passo 5 – Iteração:** a poda pode ser aplicada de forma iterativa, em pequenas etapas intercaladas por *fine-tuning*, o que costuma preservar melhor a acurácia do que uma poda agressiva em um único passo.

2.6.4 Quando a Poda é Aplicada

Tradicionalmente, a poda é aplicada após o treinamento inicial do modelo, seguida de um ajuste fino. Métodos mais recentes aplicam a poda de forma gradual durante o próprio treinamento, intercalando passos de treino e poda, o que permite à rede adaptar-se progressivamente à nova estrutura.

Uma linha de pesquisa mais recente investiga a possibilidade de podar a rede antes do treinamento, identificando sub-redes inerentemente promissoras dentro de uma rede densa inicializada aleatoriamente. Essa abordagem é impulsionada pela chamada *Lottery Ticket Hypothesis* (FRANKLE; CARBIN, 2019), segundo a qual, dentro de uma rede grande inicializada aleatoriamente, existe uma sub-rede pequena (o “bilhete premiado”) que, quando treinada isoladamente com a mesma inicialização, atinge acurácia comparável à da rede completa.

3 Trabalhos Relacionados

Este capítulo apresenta trabalhos da literatura diretamente relacionados ao tema desta pesquisa, com foco em estratégias de poda neural e, em especial, naquelas que visam a redução do consumo energético de modelos de *Deep Learning*. Para cada trabalho, são descritos os objetivos principais, a metodologia adotada e os principais resultados obtidos.

3.1 Designing Energy-Efficient Convolutional Neural Networks using Energy-Aware Pruning

Yang, Chen e Sze (2017) propõem um algoritmo de poda para redes neurais convolucionais (CNNs) cujo foco é minimizar o consumo real de energia, e não apenas o número de parâmetros ou de operações, preservando a acurácia. Os autores consideram especialmente o custo computacional associado a operações de multiplicação e acumulação (MAC) e o custo de acesso à memória em dispositivos periféricos, como *smartphones*.

O método proposto aplica poda camada por camada, priorizando aquelas cujo consumo energético estimado é maior. Para isso, os autores desenvolvem um *framework* de estimativa que ordena as camadas pela combinação de custo computacional (número de MACs) e custo de acesso à memória. Em cada camada podada, parte dos pesos é removida; em seguida, alguns pesos são restaurados para minimizar o erro no mapa de características de saída; por fim, os pesos remanescentes são ajustados localmente por meio de uma solução de mínimos quadrados. Após o processamento de todas as camadas, a rede passa por um *fine-tuning* global utilizando *backpropagation*.

Os autores reportam redução de $3,7\times$ no consumo de energia da AlexNet e $1,6\times$ no da GoogLeNet, com perda de acurácia inferior a 1%. O trabalho também demonstra empiricamente que as camadas convolucionais são as que mais consomem energia, principalmente devido ao volume de acessos à memória, e que a simples redução do número de classes tem impacto limitado na redução do consumo energético, uma vez que as camadas convolucionais não são significativamente afetadas.

A fazer: adicionar resenhas de outros trabalhos fundamentais para esta pesquisa. Sugestões: Han et al. (2015) (poda por magnitude não estruturada – seminal); Li et al. (2017) (poda estruturada L1 em filtros – seminal); Frankle e Carbin (2019) (Lottery Ticket Hypothesis); Blalock et al. (2020) (revisão crítica de métodos de poda); Liu et al. (2019) (*Rethinking the Value of Network Pruning*). Para cada um, seguir o mesmo padrão de resenha: objetivo, método, resultados principais.

4 Metodologia

Este capítulo apresenta o percurso metodológico adotado no trabalho, organizado em cinco etapas. As decisões concretas decorrentes desses critérios — incluindo o modelo selecionado, os *datasets* empregados e as configurações de poda — são descritas no Capítulo 5.

4.1 Seleção e Implementação do Modelo de Referência

O modelo de referência (*baseline*) corresponde à versão original do modelo de linguagem de grande escala (*Large Language Model* — LLM) antes de qualquer processo de poda. Ele serve como padrão para mensurar a perda de desempenho introduzida pela compressão: todas as métricas dos modelos podados são comparadas às métricas desse modelo original.

Para a seleção do modelo de referência, foram adotados os seguintes critérios:

- ser um LLM representativo, amplamente utilizado na literatura de compressão de modelos;
- dispor de pesos pré-treinados publicamente disponíveis;
- apresentar tamanho compatível com as restrições de *hardware* disponível;
- possuir ampla documentação em trabalhos de poda neural;
- contar com implementação acessível em *frameworks* consolidados, como PyTorch.

A implementação do modelo de referência utiliza a biblioteca PyTorch (PASZKE et al., 2019). Como repositório de modelos pré-treinados, é empregada a plataforma Hugging Face, um ambiente colaborativo que disponibiliza ampla variedade de modelos de linguagem de grande escala publicamente acessíveis — como, por exemplo, variantes do GPT-2. O modelo selecionado a partir dos critérios estabelecidos — incluindo a versão específica obtida via Hugging Face — é apresentado e detalhado no Capítulo 5.

4.2 Implementação das Estratégias de Poda sem Otimização

As estratégias de poda sem otimização têm por objetivo reduzir o número de parâmetros de um modelo pré-treinado, eliminando pesos ou estruturas avaliadas como menos relevantes para o desempenho preditivo. Serão avaliadas duas estratégias de poda

clássicas, escolhidas por representarem os dois extremos do espectro de granularidade da poda neural:

- poda baseada em magnitude dos pesos (HAN et al., 2015), que remove parâmetros individuais cujos valores absolutos são inferiores a um limiar; representa a granularidade mais fina possível e gera modelos esparsos sem alteração da estrutura física do modelo;
- poda estruturada (LI et al., 2017), que elimina unidades arquiteturais inteiras — como filtros convolucionais ou cabeças de atenção — reduzindo fisicamente as dimensões dos tensores e gerando modelos computacionalmente mais enxutos.

Ambas as estratégias são amplamente consolidadas na literatura de compressão de modelos neurais, com mais de uma década de aplicação e extensa validação experimental. Sua inclusão neste trabalho não constitui contribuição metodológica original; pelo contrário, sua aplicação fornece uma base de comparação estabelecida, sobre a qual serão avaliadas as estratégias otimizadas propostas na Seção 4.3.

Os critérios que orientaram a escolha das estratégias a implementar foram:

- representatividade na literatura de compressão de modelos;
- complementaridade entre granularidades distintas de remoção;
- viabilidade de implementação com as ferramentas disponíveis.

Em ambas as estratégias, são aplicadas funções de avaliação de importância de parâmetros, que orientam o processo de remoção: a poda por magnitude utiliza o valor absoluto dos pesos como critério, enquanto a poda estruturada utiliza a norma L_1 dos filtros (ou unidades arquiteturais equivalentes).

As estratégias selecionadas com base nesses critérios, bem como os detalhes de sua implementação, são apresentados no Capítulo 5.

4.3 Implementação das Estratégias de Poda com Otimização

Esta é a etapa central do trabalho. O objetivo é investigar se a combinação das estratégias de poda com técnicas de otimização resulta em modelos mais eficientes do que a poda aplicada de forma direta, sem qualquer refinamento adicional.

As estratégias de otimização a investigar incluem:

- Comparação entre taxas de poda fixas e dinâmicas;

- Variação do momento de aplicação da poda — antes do treinamento, durante ou após o *fine-tuning*;
- Análise de sensibilidade por camada, para alocação não uniforme do orçamento de poda;
- *Knowledge Distillation*, na qual um modelo menor (*student*) é treinado para reproduzir o comportamento de um modelo maior (*teacher*).

A comparação entre os modelos obtidos com e sem otimização visa identificar combinações que melhorem o balanço entre desempenho preditivo e eficiência computacional em relação às estratégias aplicadas sem otimização.

4.4 Métricas de Avaliação

As métricas de avaliação são organizadas em três dimensões: desempenho preditivo, custo computacional e impacto ambiental. Para cada métrica, descreve-se o que ela representa e como é obtida operacionalmente.

Desempenho preditivo

A métrica de desempenho preditivo adotada depende da natureza da tarefa do modelo selecionado. Em tarefas de classificação, a métrica usual é a acurácia — definida como a proporção de predições corretas sobre o total de amostras avaliadas — ou medidas derivadas, como precisão, revocação e F1 (GOODFELLOW; BENGIO; COURVILLE, 2016). Em modelos de linguagem voltados à geração de texto, a métrica padrão é a *perplexity*, que quantifica o grau de “surpresa” do modelo diante de cada *token* do conjunto de validação, constituindo uma medida da qualidade da modelagem probabilística da linguagem. Valores mais baixos de *perplexity* indicam melhor capacidade preditiva.

A métrica específica adotada neste trabalho será apresentada no Capítulo 5, em coerência com a tarefa do modelo selecionado.

Custo computacional

O custo computacional é avaliado por meio de quatro indicadores complementares:

- **Número de parâmetros remanescentes:** contagem direta dos tensores de pesos não-zerados (no caso de poda não estruturada) ou existentes (no caso de poda estruturada) no modelo após a aplicação da técnica;

- **Operações de ponto flutuante (FLOPs):** contagem das operações matemáticas necessárias para uma inferência completa, obtida por análise estática do grafo computacional do modelo. Bibliotecas específicas — como `thop`, em ambiente PyTorch — automatizam essa contagem percorrendo cada camada do modelo e acumulando as operações correspondentes;
- **Tempo de inferência:** medido empiricamente, registrando o tempo decorrido durante o processamento de uma entrada representativa em condições controladas de *hardware*. Múltiplas medições são agregadas por meio de média e desvio-padrão para mitigar a variância natural de execução;
- **Tamanho do modelo:** avaliado em duas dimensões — espaço ocupado em disco após serialização do estado do modelo e consumo de memória durante a inferência, este último registrado por ferramentas de *profiling* do PyTorch.

Impacto ambiental

O impacto ambiental é mensurado por meio de dois indicadores:

- **Consumo energético:** estimado por meio de ferramentas de monitoramento do uso de *hardware* (CPU e GPU) durante a execução dos experimentos. A biblioteca CodeCarbon (COURTY et al., 2023) integra leituras periódicas de consumo elétrico dos componentes, registrando a energia total consumida (em kWh) durante o intervalo de inferência;
- **Emissões equivalentes de CO₂:** calculadas a partir do consumo energético registrado, multiplicado por um fator de intensidade carbônica da matriz energética da região onde os experimentos são executados. A biblioteca CodeCarbon mantém base atualizada desses fatores e realiza essa conversão automaticamente.

5 Desenvolvimento

O Capítulo 4 apresentou os critérios que orientam o percurso metodológico deste trabalho. Este capítulo descreve as decisões concretas que decorrem desses critérios e o experimento tal como foi efetivamente implementado: o corpus e o modelo selecionados, quais parâmetros são considerados podáveis, como cada estratégia de poda foi implementada, como o perfil de sensibilidade é levantado e convertido em taxas por camada, o protocolo de *fine-tuning* de recuperação, o *framework* e as bibliotecas empregadas, a organização do *pipeline* experimental, as métricas coletadas, o ambiente de execução e as limitações reconhecidas.

5.1 Visão Geral

A pergunta que orienta o trabalho é empírica e comparativa. Não se pretende demonstrar que a poda comprime modelos — resultado já consolidado pela literatura (HAN et al., 2015; LI et al., 2017; BLALOCK et al., 2020) —, mas responder a duas questões mais específicas, sob condições controladas e reproduzíveis:

- (A) Em que medida a redução *teórica* de parâmetros se converte em redução *real* de tempo de inferência e de consumo de energia, e como essa conversão depende da granularidade da poda?
- (B) Uma alocação não uniforme do orçamento de poda, guiada por uma análise de sensibilidade por camada, melhora o compromisso entre compressão e qualidade em relação às alocações ingênuas?

Responder a essas duas perguntas exige que modelos comprimidos por estratégias distintas sejam avaliados exatamente pelo mesmo conjunto de métricas, no mesmo corpus e no mesmo *hardware*. Essa simetria de avaliação é condição necessária para que qualquer comparação seja interpretável, e é o princípio que organiza todo o desenvolvimento descrito adiante.

5.1.1 As Quatro Etapas Centrais

O experimento é organizado em quatro etapas sequenciais, cada uma com papel distinto.

A **primeira etapa** estabelece o modelo de referência. O GPT-2 *small* pré-treinado é carregado sem qualquer modificação e avaliado no conjunto de teste do WikiText-

2, registrando-se o vetor completo de métricas — perplexity, contagem de parâmetros, tamanho em memória, FLOPs, tempo de inferência e consumo de energia. Esse modelo denso é o referencial absoluto contra o qual todas as configurações comprimidas serão comparadas.

A **segunda etapa** aplica as duas estratégias clássicas de poda, de forma independente e sempre a partir do modelo denso original: a poda não estruturada por magnitude (HAN et al., 2015) e a poda estruturada por norma L_1 (LI et al., 2017), esta última adaptada ao *Transformer*. Para cada estratégia, varre-se toda a faixa de esparsidade, de 10% a 90% em passos de 10 pontos percentuais, e sob dois escopos de alocação distintos. A escolha dessas duas estratégias é deliberada: elas ocupam os dois extremos do espectro de granularidade e, como se verá, comportam-se de forma oposta quanto ao impacto real sobre o *hardware*.

A **terceira etapa** introduz a estratégia otimizada proposta, que constitui a contribuição central do trabalho: a análise de sensibilidade por camada. Ela se divide em duas fases. Na primeira, levanta-se o perfil de sensibilidade do modelo, podando uma camada de cada vez e medindo a degradação resultante. Na segunda, esse perfil é convertido em uma alocação não uniforme de taxas por camada, que é então avaliada nos mesmos níveis de orçamento das varreduras ingênuas.

A **quarta etapa** aplica o *fine-tuning* de recuperação. Todas as etapas anteriores operam em regime *one-shot*, isto é, podam e medem sem retreinamento, de modo a isolar o dano causado pela poda. Esta etapa reintroduz o retreinamento — como fazem tanto Han et al. (2015) quanto Li et al. (2017) — e mede quanto da degradação é recuperável.

5.1.2 Discussão das Decisões Metodológicas

Todo desenho experimental implica escolhas que condicionam o escopo dos resultados. A Tabela 1 sintetiza as principais decisões adotadas, as alternativas consideradas e a justificativa de cada uma. As seções seguintes desenvolvem em maior profundidade as que são mais críticas para a interpretação dos resultados.

5.2 Dataset

5.2.1 O WikiText-2

O WikiText-2 é um corpus de língua inglesa extraído de artigos verificados da Wikipédia, amplamente utilizado como referência na avaliação de modelos de linguagem. Diferentemente de corpora construídos a partir de frases isoladas, o WikiText preserva a estrutura original dos artigos — parágrafos completos, pontuação e capitalização mantidas

Tabela 1 – Síntese das principais decisões metodológicas do trabalho.

Decisão	Alternativas consideradas	Escolha	Justificativa
<i>Dataset</i>	WikiText-103, The Pile, OpenWebText	WikiText-2	Referência consolidada para perplexity de modelos de linguagem, com custo de avaliação compatível com o <i>hardware</i> disponível.
Modelo	GPT-2 <i>medium/large</i> , BERT, modelos da família LLaMA	GPT-2 <i>small</i>	Arquitetura <i>Transformer</i> representativa, com pesos públicos e ampla presença na literatura de compressão; 124M parâmetros cabem na GPU disponível.
Estratégias de poda	Três ou mais estratégias	Duas (magnitude e estruturada)	Cobrem os dois extremos de granularidade; restrição de tempo experimental inviabilizaria uma terceira com a mesma profundidade.
Regime de poda	Iterativo (podar e retreinar em ciclos)	<i>One-shot</i> primeiro	Isola o efeito da poda do efeito do retreinamento; o <i>fine-tuning</i> entra depois, como etapa separada e mensurável.
Estratégia otimizada	Destilação de conhecimento, quantização	Sensibilidade por camada	Ataca diretamente a hipótese do trabalho — a de que as camadas não são igualmente importantes — sem alterar a natureza da poda.
Métrica principal	<i>Accuracy</i> em tarefas descendentes	Perplexity	Métrica intrínseca padrão para modelos de linguagem, mensurável sem <i>fine-tuning</i> em tarefa específica.
Ambiente	Máquina local, Google Colab	Kaggle (GPU T4)	Acesso gratuito a GPU dedicada com sessões longas o bastante para as varreduras completas.

Fonte: *Elaboração própria.*

—, o que o torna adequado para medir a capacidade de um modelo de acompanhar contexto ao longo de textos extensos.

Emprega-se especificamente a variante `wikitext-2-raw-v1`, na qual o texto não sofre substituição de palavras raras por um marcador genérico. Essa escolha é neces-

sária porque o GPT-2 utiliza tokenização por *subwords*, capaz de representar palavras fora do vocabulário decompondo-as em fragmentos; a variante processada anularia essa capacidade e distorceria a perplexity medida.

5.2.2 Justificativa da Escolha

Três fatores motivaram a escolha. O primeiro é a comparabilidade: perplexities medidas no WikiText-2 são reportadas em grande número de trabalhos, o que permite verificar se o *baseline* obtido é coerente com o esperado — verificação efetivamente realizada, como se discute na Seção 6.1. O segundo é o custo: o conjunto de teste é pequeno o suficiente para que uma avaliação completa de perplexity leve menos de um minuto na GPU utilizada, o que é determinante em um trabalho que executa mais de uma centena de avaliações. O terceiro é a existência de uma divisão oficial em treino, validação e teste, que dispensa decisões arbitrárias de particionamento.

Registre-se que o corpus é parametrizável na implementação: a substituição pelo WikiText-103, de mesma natureza e cerca de cinquenta vezes maior, exige apenas a alteração de um argumento de linha de comando. Essa flexibilidade foi mantida deliberadamente para que trabalhos futuros possam verificar a generalidade dos resultados sem reescrever o código.

5.2.3 Pré-processamento e Tokenização

O pré-processamento é intencionalmente mínimo, para que a perplexity medida reflita o corpus e não decisões de limpeza. As linhas do conjunto são unidas por quebra dupla, preservando a separação entre parágrafos tal como no corpus original, e o texto resultante é tokenizado como um fluxo contínuo único.

A tokenização emprega o tokenizador original do GPT-2, baseado no algoritmo BPE (*Byte Pair Encoding*), com vocabulário de 50.257 *tokens*. Manter o tokenizador original é indispensável: os *embeddings* do modelo foram aprendidos para esse vocabulário específico, e qualquer substituição invalidaria os pesos pré-treinados.

5.2.4 Divisão dos Dados

Adota-se a divisão oficial do corpus. O **conjunto de teste** é utilizado em todas as medições de perplexity reportadas, tanto do modelo denso quanto de todas as configurações comprimidas. O **conjunto de treinamento** é utilizado exclusivamente na etapa de *fine-tuning* de recuperação (Seção 5.6). O **conjunto de validação** é empregado apenas para acompanhar a evolução do treinamento ao longo dos passos, quando essa curva é solicitada, e nunca para selecionar resultados reportados.

Essa separação estrita importa: como as varreduras *one-shot* não realizam treinamento algum, elas não têm contato com o conjunto de treinamento, e a perplexity de teste é uma medida limpa do dano causado pela poda.

5.3 Modelo de Referência

5.3.1 O GPT-2 *small*

O modelo de referência é o GPT-2 *small*, proposto por Radford et al. (2019) e descrito na Seção 2.3.5. Trata-se de um modelo autorregressivo composto exclusivamente pelo componente decodificador da arquitetura *Transformer*. Os pesos pré-treinados são obtidos pela plataforma Hugging Face, que os disponibiliza publicamente, e carregados por meio da biblioteca PyTorch (PASZKE et al., 2019).

A Tabela 2 resume a configuração da variante utilizada.

Tabela 2 – Configuração do GPT-2 *small* utilizado como modelo de referência.

Característica	Valor
Blocos <i>Transformer</i> empilhados	12
Cabeças de atenção por bloco	12
Dimensão do modelo (d_{model})	768
Dimensão intermediária do MLP (d_{ff})	3.072
Tamanho do vocabulário	50.257
Janela de contexto máxima	1.024 <i>tokens</i>
Total de parâmetros	124.439.808
Tamanho em memória	474,70 MB
FLOPs por janela de 512 <i>tokens</i>	126,6 G
Perplexity no WikiText-2 (teste)	25,17

Fonte: Elaboração própria.

5.3.2 Justificativa da Escolha

O GPT-2 *small* atende simultaneamente aos cinco critérios estabelecidos na Seção 4.1. É um modelo de linguagem representativo e amplamente estudado; dispõe de pesos pré-treinados de acesso público; possui 124 milhões de parâmetros, volume que cabe confortavelmente na memória da GPU disponível mesmo durante o *fine-tuning*; é frequentemente empregado como objeto de estudo em trabalhos de compressão; e conta com implementação madura em PyTorch.

Cabe observar que, embora seja a menor variante da família GPT-2, o modelo é cerca de cinco vezes maior que a ResNet-50, arquitetura tradicionalmente usada em estudos de poda em visão computacional. Essa diferença de escala é justamente o que

torna o trabalho pertinente: as conclusões da literatura de poda foram estabelecidas majoritariamente em redes convolucionais, e não é evidente que se transfiram para modelos com esse perfil de parâmetros.

5.3.3 Parâmetros Podáveis

Uma decisão de implementação que condiciona todos os resultados é a definição de quais parâmetros são elegíveis à poda. Adotam-se, por convenção consolidada na literatura, apenas as matrizes de peso das projeções lineares internas aos blocos *Transformer*, a saber, quatro por bloco:

- `attn.c_attn` — projeção que produz consulta, chave e valor;
- `attn.c_proj` — projeção de saída da atenção;
- `mlp.c_fc` — primeira transformação linear do MLP;
- `mlp.c_proj` — segunda transformação linear do MLP.

Com 12 blocos, são 48 matrizes podáveis. Ficam deliberadamente fora do escopo da poda:

- as matrizes de *embedding* de *token* (`wte`) e de posição (`wpe`), que codificam o vocabulário e não representam computação repetida a cada camada;
- a cabeça de linguagem (`lm_head`), que no GPT-2 compartilha pesos com `wte` — podá-la corromperia os *embeddings*;
- vieses e parâmetros de normalização de camada, que são numericamente baratos e desproporcionalmente sensíveis.

Essa restrição tem consequência direta na leitura dos resultados: quando se afirma que uma configuração tem 50% de esparsidade, trata-se de 50% das matrizes podáveis, não do modelo inteiro. Como os *embeddings* do GPT-2 concentram parcela expressiva dos parâmetros, a esparsidade sobre o total é sempre menor que a esparsidade sobre o conjunto podável, e as duas grandezas são reportadas separadamente.

5.4 Métodos de Poda da Literatura

5.4.1 Método 1 — Poda Não Estruturada por Magnitude

O primeiro método segue Han et al. (2015). O critério de importância é o valor absoluto do peso: assume-se que pesos próximos de zero contribuem pouco para a saída

e podem ser anulados. Dada uma taxa de esparsidade alvo, calcula-se o limiar correspondente ao quantil apropriado da distribuição de $|w|$ e zeram-se todos os pesos abaixo dele.

A implementação preserva a característica essencial do método: os pesos são **zerados**, não removidos. As matrizes conservam suas dimensões originais e a arquitetura permanece intacta — o que, como discutido na Seção 2.6, é precisamente a razão pela qual esse método não reduz o custo de inferência em *hardware* convencional.

5.4.2 Método 2 — Poda Estruturada por Norma L_1

O segundo método adapta Li et al. (2017) ao *Transformer*. O trabalho original poda filtros convolucionais ordenados por norma L_1 ; aqui, as unidades estruturais equivalentes são as **cabeças de atenção** e os **neurônios da camada intermediária do MLP**. Para cada uma dessas estruturas calcula-se a norma L_1 dos pesos que lhe correspondem, e removem-se as de menor norma.

A diferença decisiva em relação ao método anterior é que a remoção é **física**. As matrizes são efetivamente reconstruídas com dimensões menores: as colunas de consulta, chave e valor associadas às cabeças removidas são eliminadas de `c_attn`, as linhas correspondentes saem de `c_proj`, e a contabilidade interna do módulo de atenção — número de cabeças e dimensão de divisão — é atualizada de acordo. O resultado não é um modelo esparso, mas uma arquitetura menor e densa.

A implementação admite três alvos: apenas cabeças, apenas neurônios MLP, ou ambos simultaneamente. Uma salvaguarda impede que qualquer bloco fique sem nenhuma cabeça ou sem nenhum neurônio, situação que romperia o fluxo residual e inviabilizaria o modelo.

Convém registrar que a esparsidade alvo, neste método, é expressa como fração de **estruturas** removidas, não de parâmetros. Como cabeças e neurônios têm custos diferentes em parâmetros, a fração de parâmetros efetivamente eliminada difere da taxa solicitada, e é reportada separadamente em todas as tabelas.

5.4.3 Escopos de Alocação

Ambos os métodos são avaliados sob dois escopos, que representam formas distintas de distribuir o orçamento de poda entre as camadas:

- **Global:** o critério de importância é aplicado ao modelo inteiro de uma só vez. Um único limiar — ou uma única ordenação de estruturas — decide o que sai, de modo que camadas com pesos sistematicamente menores cedem proporcionalmente mais.

- **Uniforme:** cada camada recebe exatamente a mesma fração de poda, decidida isoladamente dentro dela. É a alocação ingênua por excelência, pois ignora qualquer diferença de importância entre camadas.

O contraste entre esses dois escopos é o que estabelece a linha de base contra a qual a estratégia otimizada é julgada. Adianta-se que nenhum deles domina o outro em toda a faixa de esparsidade, e é essa constatação que motiva a análise de sensibilidade.

5.4.4 Taxas de Poda Avaliadas

Cada combinação de método e escopo é avaliada em dez níveis: 0% (o modelo denso, incluído como controle interno de cada varredura) e de 10% a 90% em passos de 10 pontos percentuais. A varredura completa da faixa é uma escolha deliberada: como não se conhecia de antemão onde estaria o ponto de colapso de cada estratégia, restringir os níveis a uma faixa estreita poderia ocultar exatamente o comportamento que se deseja caracterizar.

Cada nível parte de um modelo recarregado do zero, e não do modelo podado no nível anterior. Sem esse cuidado, a poda seria cumulativa e os níveis deixariam de ser independentes entre si.

5.5 Estratégia Otimizada: Sensibilidade por Camada

Os métodos descritos até aqui compartilham uma limitação: nenhum deles mede o quanto cada camada realmente importa. O escopo uniforme trata todas as camadas como intercambiáveis; o escopo global decide com base na magnitude dos pesos, que é um *proxy* de importância, não uma medida dela. A estratégia proposta substitui esse *proxy* por uma medição direta, em duas fases.

5.5.1 Fase 1 — Perfil de Sensibilidade

Para cada camada ℓ do modelo e para cada estratégia de poda, poda-se **exclusivamente** aquela camada a uma taxa-sonda fixa, mantendo todas as demais intactas, e mede-se a perplexity resultante. A degradação é registrada em escala logarítmica:

$$\Delta_\ell = \ln \left(\frac{\text{PPL}_\ell}{\text{PPL}_{\text{base}}} \right), \quad (5.1)$$

onde PPL_ℓ é a perplexity do modelo com apenas a camada ℓ podada. O logaritmo é necessário porque a perplexity de modelos podados sem recuperação varia por ordens de grandeza; em escala linear, uma única camada catastrófica tornaria todas as demais indistinguíveis entre si.

Adota-se taxa-sonda de 50%. O procedimento não envolve treinamento e produz, ao final, um perfil com uma medida de sensibilidade por camada, para cada estratégia. Como as duas estratégias removem coisas diferentes, os perfis são levantados separadamente e não são intercambiáveis — ponto que a Seção 6.4.3 demonstra experimentalmente.

5.5.2 Fase 2 — Alocação Adaptativa de Taxas

O perfil é então convertido em uma taxa de poda por camada. Atribui-se a cada camada uma taxa proporcional à sua robustez — o inverso da degradação medida — elevada a um expoente β e escalada por um fator único α , escolhido de modo que a média das taxas, ponderada pelo número de parâmetros podáveis de cada camada, iguale o orçamento global s pretendido:

$$r_\ell = \min\left(\alpha \cdot \Delta_\ell^{-\beta}, r_{\max}\right), \quad \frac{\sum_\ell w_\ell r_\ell}{\sum_\ell w_\ell} = s, \quad (5.2)$$

em que w_ℓ é o número de parâmetros podáveis da camada ℓ e $r_{\max} = 0,95$ é um teto que impede que mesmo a camada mais robusta seja integralmente zerada. Como a fração podada é monótona em α , o fator é obtido por bisseção.

O expoente β controla o quanto o perfil é levado a sério. Com $\beta = 0$ a regra degenera na poda uniforme; com $\beta = 1$ emprega-se o inverso puro da degradação; valores intermediários amortecem a razão entre camadas. Esse amortecimento não é um detalhe: a razão de robustez entre duas camadas é ilimitada, e sem controle uma única camada muito robusta absorveria praticamente todo o orçamento. Os experimentos avaliam $\beta = 1,0$ e $\beta = 0,5$ sobre o mesmo perfil, o que fornece a ablação da função de alocação sem exigir a repetição da fase 1, que é a etapa cara.

5.6 *Fine-tuning* de Recuperação

Todas as etapas anteriores operam em regime *one-shot*. Esse regime isola o dano causado pela poda, mas não corresponde ao procedimento da literatura: tanto Han et al. (2015) quanto Li et al. (2017) retreinam o modelo após podá-lo. Esta etapa fecha essa lacuna, medindo, para cada configuração, a perplexity antes e depois do retreinamento.

Duas decisões de implementação merecem registro por afetarem a validade dos resultados.

A primeira é o **controle denso**. O GPT-2 não foi pré-treinado no WikiText-2, de modo que o simples *fine-tuning* do modelo denso nesse corpus já reduz a perplexity por adaptação de domínio, sem poda alguma envolvida. Sem essa linha de controle, seria impossível separar a recuperação do dano da poda da mera adaptação ao corpus. O

modelo denso é, portanto, retreinado com o mesmo orçamento de passos e avaliado junto às demais configurações.

A segunda é a **preservação da esparsidade**. Na poda não estruturada os pesos são apenas zerados, e o gradiente os restauraria já no primeiro passo do otimizador; ao final do treinamento, o modelo estaria novamente denso. Como em Han et al. (2015), mantém-se uma máscara binária fixa, capturada logo após a poda e reaplicada depois de cada passo do otimizador. Registre-se que zerar o gradiente não seria suficiente: o otimizador utilizado possui momento e regularização por decaimento de pesos, e ambos alteram um parâmetro mesmo quando seu gradiente é nulo. A esparsidade medida ao final do treinamento é registrada como verificação de integridade. Na poda estruturada não há máscara, pois as estruturas já foram fisicamente removidas.

O treinamento emprega o otimizador AdamW, com aquecimento linear da taxa de aprendizado seguido de decaimento linear, recorte de norma do gradiente e acúmulo de gradiente para viabilizar lotes efetivos maiores do que a memória da GPU comportaria. O consumo de energia do treinamento é medido separadamente do consumo da inferência: trata-se de um custo pago uma única vez, que precisa ser amortizado contra a economia por inferência para que o balanço energético do trabalho feche corretamente.

Diferentemente de todas as etapas anteriores, esta é estocástica — a ordem dos lotes e o *dropout* introduzem variabilidade — e, portanto, é a única em que a semente aleatória efetivamente altera o resultado.

5.7 *Framework* de Implementação

A implementação é feita em Python, com o *framework* PyTorch (PASZKE et al., 2019) para construção, manipulação e treinamento dos modelos. O PyTorch foi escolhido por seu sistema de diferenciação automática, que constrói dinamicamente o grafo computacional e simplifica a implementação do *backpropagation* durante o *fine-tuning*, e por seu suporte nativo à aceleração em GPU.

Os pesos pré-treinados são obtidos pela plataforma Hugging Face por meio da biblioteca `transformers`. Cabe a distinção, já feita no Capítulo 4: a Hugging Face é a plataforma que hospeda e distribui os modelos, ao passo que PyTorch é a biblioteca que os executa.

As demais bibliotecas empregadas são:

- `datasets`: carregamento do corpus WikiText a partir da plataforma Hugging Face;
- `thop`: contagem de operações de ponto flutuante por análise do grafo computacional;

- **CodeCarbon** (COURTY et al., 2023): medição empírica do consumo de energia e estimativa das emissões de CO_2 , a partir do monitoramento do *hardware* combinado à intensidade de carbono da rede elétrica local.

Uma observação relevante sobre a implementação da poda: ambas as estratégias foram implementadas manualmente, sem recurso a bibliotecas dedicadas. No caso da poda estruturada, essa decisão foi imposta pelo ambiente — as versões mais recentes da biblioteca **transformers** removeram a interface de poda de cabeças que existia em versões anteriores. A remoção física das estruturas é, portanto, realizada diretamente: as matrizes são reconstruídas com as dimensões reduzidas e a contabilidade interna do módulo de atenção é atualizada. A implementação foi validada por testes que verificam, entre outras propriedades, a equivalência numérica entre a remoção física das estruturas e o simples anulamento das linhas correspondentes.

5.8 Pipeline Experimental

O código está organizado em módulos de responsabilidade única, o que permite que cada estratégia de poda reutilize exatamente a mesma rotina de avaliação — condição para que as comparações sejam legítimas. A Tabela 3 descreve os módulos e a Figura 7 representa o fluxo experimental.

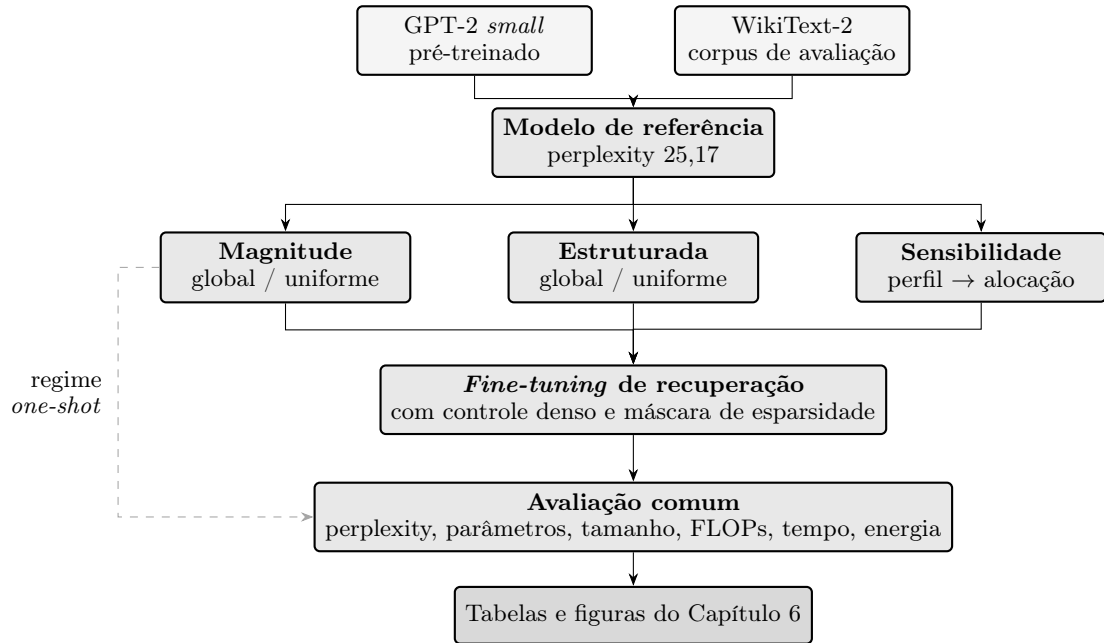
Tabela 3 – Organização modular da implementação.

Módulo	Responsabilidade
<code>config</code>	Configuração central: modelo, corpus, parâmetros de avaliação, semente e ambiente.
<code>data</code>	Carregamento e tokenização do corpus.
<code>model</code>	Carregamento do GPT-2 pré-treinado.
<code>eval</code>	Perplexity por janela deslizante e métricas de custo computacional.
<code>energy</code>	Medição de energia e emissões.
<code>prune_magnitude</code>	Poda não estruturada por magnitude, nos dois escopos.
<code>prune_structured</code>	Poda estruturada de cabeças e neurônios, com remoção física.
<code>sensitivity</code>	Perfil de sensibilidade (fase 1) e alocação adaptativa (fase 2).
<code>finetune</code>	<i>Fine-tuning</i> de recuperação, com preservação de esparsidade.

Fonte: Elaboração própria.

Como indica a linha tracejada da figura, as varreduras *one-shot* contornam a etapa de *fine-tuning* e vão direto à avaliação. Essa bifurcação é o que permite separar, nos resultados, o dano causado pela poda daquilo que o retreinamento consegue recuperar.

Figura 7 – Fluxo experimental do trabalho. Todas as estratégias convergem para a mesma rotina de avaliação, o que garante a simetria necessária às comparações.



Fonte: Elaborada pelo autor.

5.9 Métricas de Avaliação

Toda configuração — o modelo denso, as podadas pelos métodos da literatura e as podadas pela estratégia otimizada — é avaliada pelo mesmo conjunto de métricas, cobrindo as três dimensões definidas na Seção 4.4. A Tabela 4 as resume.

Tabela 4 – Métricas de avaliação coletadas para cada configuração.

Métrica	O que mede	Unidade	Ferramenta
Perplexity	Qualidade da modelagem da linguagem; quanto menor, melhor	—	PyTorch
Parâmetros totais	Parâmetros existentes na arquitetura	unidades	PyTorch
Parâmetros não nulos	Parâmetros efetivamente diferentes de zero	unidades	PyTorch
Esparsidade real	Fração de parâmetros podáveis eliminados	%	PyTorch
Tamanho do modelo	Memória ocupada pelos parâmetros	MB	PyTorch
FLOPs	Operações de ponto flutuante por inferência	FLOPs	thop
Tempo de inferência	Tempo médio de uma passagem direta	ms	perf_counter
Energia consumida	Energia gasta durante a avaliação	kWh	CodeCarbon
Emissões de CO_2	Emissões equivalentes ao consumo	kg	CodeCarbon

Fonte: Elaboração própria.

5.9.1 Perplexity

A perplexity é a métrica de desempenho preditivo adotada. Ela é calculada pelo método da janela deslizante, padrão para modelos autorregressivos: o fluxo de *tokens* do conjunto de teste é percorrido em janelas do tamanho máximo de contexto do modelo, com passo menor que a janela, de modo que cada *token* seja previsto dispondo de contexto anterior substancial. Apenas os *tokens* ainda não avaliados contribuem para a perda em cada janela; os demais entram como contexto.

Emprega-se janela de 1.024 *tokens* com passo de 512. Passos menores produzem estimativas mais precisas ao custo de mais computação; o valor adotado é o compromisso usual na literatura, e é mantido idêntico em todas as configurações avaliadas.

5.9.2 Custo Computacional

A contagem de **parâmetros** é reportada em duas formas complementares. O total reflete o que existe na arquitetura; a contagem de não nulos reflete o que sobrou depois da poda. Na poda não estruturada as duas divergem — há zeros, mas a arquitetura é a mesma —, enquanto na poda estruturada elas coincidem, pois o que foi removido deixou de existir. Essa divergência é, por si só, um resultado: ela evidencia a diferença entre compressão contábil e compressão física.

Os **FLOPs** são contabilizados por análise do grafo computacional. Registre-se uma particularidade da implementação: as projeções internas do GPT-2 não são camadas lineares convencionais, e sim um tipo específico de camada que a ferramenta de contagem não reconhece por padrão. Sem tratamento, a contagem ignoraria justamente as matrizes que a poda modifica, e a métrica sairia constante. Foi necessário, portanto, registrar uma regra de contagem própria para esse tipo de camada. Reconhece-se uma limitação remanescente: as multiplicações de matriz internas ao mecanismo de atenção não são contabilizadas pela ferramenta em nenhuma configuração. Como a omissão é idêntica para todas as estratégias, as comparações permanecem válidas.

O **tempo de inferência** é medido sobre uma entrada representativa de 512 *tokens*, com execuções de aquecimento descartadas — para mitigar efeitos de inicialização da GPU — seguidas de execuções cronometradas, das quais se reporta média e desvio-padrão.

5.9.3 Impacto Ambiental

O consumo de **energia** é medido com o CodeCarbon, que monitora o *hardware* durante a execução do bloco de avaliação e reporta a energia consumida em quilowatts-hora. As **emissões de CO₂** são estimadas a partir desse consumo e da intensidade de carbono da rede elétrica onde o código executa.

Sobre esta última métrica cabe uma ressalva importante, identificada durante os experimentos: a intensidade de carbono utilizada varia conforme a sessão de execução. Em consequência, as emissões não são comparáveis entre execuções realizadas em sessões distintas, e a análise dos resultados apoia-se no consumo de energia, não nas emissões derivadas dele.

5.10 Ambiente de Execução e Reprodutibilidade

Os experimentos são executados na plataforma Kaggle, em sessões com GPU NVIDIA Tesla T4. A implementação detecta automaticamente o dispositivo disponível, o que permite executar localmente em CPU para testes rápidos sem alteração de código.

Quanto à reprodutibilidade, três aspectos merecem registro.

O primeiro é que as varreduras *one-shot* são **determinísticas**. Não há treinamento, o modelo opera em modo de avaliação, o conjunto de teste é fixo e os critérios de poda são ordenações determinísticas sobre pesos fixos. Verificou-se empiricamente que execuções independentes, em sessões distintas, reproduzem a perplexity até a última casa decimal. Repetições com sementes diferentes, nessas etapas, produziriam linhas idênticas e não acrescentariam informação. A semente passa a importar apenas no *fine-tuning*.

O segundo é que as medições de **tempo e energia dependem da sessão**. Observou-se que o mesmo modelo denso, sem qualquer poda, registrou tempos e consumos sensivelmente distintos em sessões diferentes da plataforma, o que se atribui à variação de *hardware* físico e de carga concorrente. Por essa razão, comparações de custo entre varreduras executadas em sessões distintas são sempre feitas em termos relativos ao modelo denso da própria sessão, nunca em valores absolutos.

O terceiro é que cada configuração experimental grava uma linha em arquivo de resultados, contendo não apenas as métricas mas também os parâmetros que a geraram — estratégia, escopo, taxa alvo, taxa efetiva e, no caso da estratégia otimizada, as taxas atribuídas a cada camada. Isso permite rastrear qualquer número apresentado no Capítulo 6 até a configuração exata que o produziu.

5.11 Limitações Metodológicas

Reconhecem-se, desde o desenho experimental, limitações que condicionam o escopo dos resultados:

- **Um único modelo e um único corpus.** Os experimentos empregam exclusivamente o GPT-2 *small* sobre o WikiText-2. Os resultados podem não se transferir diretamente para modelos de escala substancialmente maior ou para corpora de

natureza distinta. A implementação foi mantida parametrizável justamente para viabilizar essa verificação em trabalhos futuros.

- **Duas estratégias de poda, e não mais.** A restrição de tempo experimental levou à escolha de duas estratégias que cobrem os extremos de granularidade. Métodos baseados em informação de gradiente ou de segunda ordem ficaram fora do escopo.
- **Taxa-sonda única no perfil de sensibilidade.** O perfil é levantado a uma única taxa. Nada garante que a ordenação de sensibilidade entre camadas se mantenha em regimes de poda muito mais brandos ou muito mais agressivos.
- **Sensibilidade medida em isolamento.** A sonda mede o efeito de cada camada isoladamente e ignora interações entre elas. Como se verifica no Capítulo 6, a degradação é fortemente superaditiva, de modo que o perfil deve ser lido como ordenação relativa de importância, e não como modelo preditivo do desempenho de uma configuração completa.
- **Dependência do *hardware*.** Tempo e energia são intrinsecamente dependentes da máquina. Os valores absolutos não devem ser comparados entre ambientes distintos; a análise apoia-se em comparações relativas dentro de um mesmo ambiente.
- **Hiperparâmetros não otimizados exaustivamente.** Os hiperparâmetros do *fine-tuning* foram definidos a partir da literatura e de experimentos preliminares, sem busca sistemática. Valores melhores podem existir.
- **Estimativa de emissões.** A conversão de energia em emissões depende de tabelas regionais de intensidade de carbono, que carregam incerteza e variaram entre as sessões de execução. As emissões devem ser lidas como aproximações de ordem de grandeza.

Apesar dessas limitações, o desenho descrito é suficiente para responder às duas perguntas enunciadas na Seção 5.1, que são o objeto do Capítulo 6.

6 Resultados

Este capítulo apresenta os resultados experimentais obtidos com o GPT-2 *small* sobre o WikiText-2. A exposição segue a ordem em que os experimentos foram conduzidos: primeiro as duas estratégias clássicas de poda — magnitude (Seção 6.2) e estruturada (Seção 6.3) —, cujos resultados delimitam o problema; em seguida a estratégia otimizada proposta, em suas duas fases: o levantamento do perfil de sensibilidade por camada (Seção 6.4) e a varredura da alocação dele derivada (Seção 6.5). A Seção 6.6 sintetiza os achados.

6.1 Protocolo e Modelo de Referência

Todos os experimentos partem do GPT-2 *small* pré-treinado (124.439.808 parâmetros), avaliado no conjunto de teste do WikiText-2 por janela deslizante com *stride* de 512 *tokens*. A perplexity do modelo denso é de **25,17**, valor coerente com o reportado na literatura para essa configuração, o que valida a implementação do procedimento de avaliação. As medições de custo do modelo denso são: 474,70 MB de memória de parâmetros e 126,6 GFLOPs por janela de 512 *tokens*.

Duas ressalvas metodológicas condicionam a leitura das tabelas que se seguem.

A primeira é que **todos os resultados deste capítulo são *one-shot***: poda-se o modelo pré-treinado e mede-se imediatamente, sem qualquer *fine-tuning* de recuperação. Essa escolha isola o efeito da poda do efeito do retreinamento, mas implica que as perplexities aqui reportadas são substancialmente piores que as da literatura, em que a poda é seguida de retreinamento. As curvas não devem, portanto, ser lidas como proposta de configuração de uso, e sim como medida do dano que cada estratégia de alocação causa por si só.

A segunda é que as varreduras foram executadas em sessões distintas do ambiente Kaggle, e as medições de **custo dependem da sessão**. O mesmo modelo denso registrou 40,5 ms por janela em uma sessão, 54,0 ms em outra e 46,3 ms em uma terceira; o consumo de energia do modelo denso variou de forma análoga, entre 1176 e 1572 μWh . Por essa razão, comparações de tempo e energia **entre** varreduras são sempre apresentadas em termos relativos ao *baseline* da própria sessão, nunca em valores absolutos. As emissões de CO₂, que dependem adicionalmente do fator de carbono da rede elétrica do *datacenter* — que também variou entre sessões —, não são comparadas entre varreduras.

6.2 Poda por Magnitude

A Tabela 5 apresenta a varredura da poda por magnitude (HAN et al., 2015) nos dois escopos de alocação implementados: *global*, em que um único limiar sobre o valor absoluto dos pesos é aplicado a toda a rede, e *uniforme*, em que cada camada recebe a mesma fração de poda, com limiar próprio.

Tabela 5 – Poda por magnitude *one-shot* no GPT-2 *small* / WikiText-2. As colunas de custo referem-se ao escopo uniforme; o escopo global produz valores idênticos, pois nenhum dos dois altera a arquitetura.

Esparsidade	Perplexity		Custo		
	Global	Uniforme	Params. $\neq 0$	Tamanho	FLOPs
0 %	25,17	25,17	124,4 M	474,7 MB	126,6 G
10 %	25,40	25,47	115,9 M	474,7 MB	126,6 G
20 %	26,87	27,22	107,5 M	474,7 MB	126,6 G
30 %	40,23	36,42	99,0 M	474,7 MB	126,6 G
40 %	165,20	104,82	90,5 M	474,7 MB	126,6 G
50 %	1096,50	473,28	82,0 M	474,7 MB	126,6 G
60 %	5121,25	2004,20	73,5 M	474,7 MB	126,6 G
70 %	9371,16	6891,85	65,0 M	474,7 MB	126,6 G
80 %	8847,27	10871,00	56,5 M	474,7 MB	126,6 G
90 %	4612,69	13996,88	48,0 M	474,7 MB	126,6 G

O resultado mais importante da tabela está nas três últimas colunas, e é negativo: **o custo não cai**. Entre 0 % e 90 % de esparsidade, o número de parâmetros não nulos cai de 124,4 M para 48,0 M — uma redução de 61 % —, enquanto o tamanho em memória, os FLOPs e o tempo de inferência permanecem rigorosamente constantes. A explicação é direta: a poda não estruturada substitui pesos por zeros, mas não remove nada da arquitetura. Os tensores mantêm as mesmas dimensões, e a GPU multiplica os zeros exatamente como multiplicava os valores originais. O ganho de 61 % em parâmetros é contábil, não físico.

Esse achado responde empiricamente, e pela negativa, à primeira das duas perguntas que orientam este trabalho — a de se a redução teórica de parâmetros se converte em redução real de consumo. Na Tesla T4 utilizada, e sem suporte de *hardware* ou de *kernels* para esparsidade, a poda não estruturada não entrega ganho de eficiência algum. A pequena tendência de queda na energia medida (de 1572 para 1432 μWh) é da ordem da variação observada entre execuções repetidas do mesmo modelo denso e não deve ser interpretada como efeito da poda.

Quanto à qualidade, a curva de perplexity apresenta um joelho acentuado em torno de 30 %. Até 20 % de esparsidade a poda é praticamente gratuita (acréscimo de 0,3

a 2,1 pontos de perplexity); a partir de 40 % a degradação torna-se explosiva. A região utilizável em regime *one-shot* é, portanto, limitada a esparsidades de até 20 %.

Por fim, a comparação entre os dois escopos revela um cruzamento duplo: os escopos empatam até 20 %, o uniforme domina claramente entre 30 % e 70 % — exatamente a faixa em que a escolha de alocação teria consequência prática — e o global só apresenta valores menores em 80 % e 90 %, região em que ambos os modelos já estão destruídos (perplexity acima de 4600) e a comparação perde sentido. Na faixa que importa, portanto, a alocação uniforme é preferível à global. Esse é um resultado contraintuitivo: o escopo global é o que aloca o orçamento com base em uma medida de importância — a magnitude do peso — e ainda assim perde para a alocação que ignora qualquer medida de importância. Trata-se do primeiro indício de que a magnitude do peso é um *proxy* pobre para a importância de um parâmetro.

6.3 Poda Estruturada

A Tabela 6 apresenta a varredura da poda estruturada (LI et al., 2017), adaptada ao *Transformer*: remoção física de cabeças de atenção e de neurônios das camadas MLP, ordenados por norma L_1 . A esparsidade-alvo é expressa como fração de *estruturas* removidas; a fração de parâmetros efetivamente eliminada aparece como esparsidade real.

Tabela 6 – Poda estruturada *one-shot* (cabeças e neurônios MLP, escopo global) no GPT-2 *small* / WikiText-2. O tempo de inferência é normalizado pelo *baseline* da mesma sessão (40,5 ms).

Estrut.	Perplexity	Params.	Tamanho	FLOPs	Tempo rel.	Energia rel.
0 %	25,17	124,4 M	474,7 MB	126,6 G	1,000	1,000
10 %	356,20	116,0 M	442,6 MB	118,0 G	0,968	0,975
20 %	2102,06	107,6 M	410,5 MB	109,3 G	0,908	0,945
30 %	2413,52	99,0 M	377,6 MB	100,5 G	0,831	0,865
40 %	2434,92	90,6 M	345,5 MB	91,9 G	0,756	0,786
50 %	2402,58	81,9 M	312,6 MB	83,1 G	0,688	0,714
60 %	2957,26	73,5 M	280,5 MB	74,5 G	0,628	0,638
70 %	4635,36	65,1 M	248,3 MB	65,9 G	0,557	0,558
80 %	7620,02	56,5 M	215,5 MB	57,0 G	0,503	0,489
90 %	5822,89	48,1 M	183,3 MB	48,4 G	0,445	0,413

O contraste com a Tabela 5 é o achado central deste trabalho no plano da eficiência. Aqui **o custo cai, e cai proporcionalmente aos parâmetros**: a 90 % de estruturas removidas, o tempo de inferência cai 56 %, a energia 59 %, o tamanho em memória 61 % e os FLOPs 62 %, acompanhando a redução de 61 % no número de parâmetros. As duas tabelas formam, juntas, a resposta completa à primeira pergunta empírica:

a redução de parâmetros só se converte em redução de consumo quando a poda remove estruturas inteiras da arquitetura, e não quando apenas as zera.

O preço dessa eficiência, contudo, é severo em regime *one-shot*. Já com 10 % de estruturas removidas a perplexity salta de 25,17 para 356,20 — compare-se com 25,40 na poda por magnitude ao mesmo nível. **Não existe região gratuita** na poda estruturada sem retreinamento, o que é coerente com o procedimento original de Li et al. (2017), em que a poda é sempre seguida de *fine-tuning*.

Dois resultados secundários merecem registro. O primeiro é que a inversão de escopo observada na magnitude se repete ao contrário: na poda estruturada é o escopo **global** que domina, e a alocação uniforme colapsa em taxas altas (perplexity de 53.313 a 80 % e de 827.316 a 90 %, contra 7620 e 5823 do global). Somado ao achado da seção anterior, isso estabelece que **nenhuma das duas alocações ingênuas vence nos dois regimes** — a constatação que motiva diretamente a estratégia otimizada.

O segundo é a assimetria entre os dois tipos de estrutura. Remover cabeças de atenção é notavelmente mais tolerável que remover neurônios MLP: com 129 das 144 cabeças removidas (90 %), a perplexity fica em 781, ao passo que a remoção equivalente de neurônios MLP produz valores acima de 3900. A curva das cabeças é, ainda, não monotônica, com mínimo local em torno de 30 % (perplexity 162, inferior aos 237 obtidos com apenas 10 % removidas) — eco direto do achado de Michel, Levy e Neubig (2019) sobre a redundância das cabeças de atenção. Em contrapartida, as camadas MLP concentram cerca de dois terços dos parâmetros podáveis, de modo que a poda restrita a cabeças tem alcance limitado como estratégia de compressão.

6.4 Perfil de Sensibilidade por Camada

As duas seções anteriores mostraram que nenhuma alocação ingênuas do orçamento de poda domina o espectro de esparsidade: na poda por magnitude o escopo uniforme prevaleceu na faixa útil (Seção 6.2), enquanto na poda estruturada foi o escopo global que obteve as melhores perplexidades (Seção 6.3) — e ambas as alocações colapsam em algum regime. A hipótese central deste trabalho é que a causa é comum aos dois casos: as camadas do *Transformer* não são igualmente importantes, de modo que tanto tratá-las de forma idêntica (alocação uniforme) quanto confiar exclusivamente na magnitude dos pesos como proxy de importância (alocação global) distribuem mal o orçamento disponível.

Esta seção reporta o experimento que testa diretamente essa hipótese. Para cada uma das 12 camadas do GPT-2 *small* e para cada uma das duas estratégias, poda-se **apenas** aquela camada a uma taxa-sonda fixa de 50 %, mantendo-se as demais intactas, e mede-se a perplexity resultante no conjunto de teste do WikiText-2. Não há *fine-tuning*

— o procedimento é *one-shot*, como nas varreduras anteriores. A degradação é registrada em escala logarítmica,

$$\Delta_\ell = \ln\left(\frac{\text{PPL}_\ell}{\text{PPL}_{\text{base}}}\right), \quad (6.1)$$

onde PPL_ℓ é a perplexity do modelo com apenas a camada ℓ podada e $\text{PPL}_{\text{base}} = 25,17$ é a perplexity do modelo denso. O logaritmo é necessário porque a perplexity de modelos podados sem recuperação varia por ordens de grandeza: em escala linear, uma única camada catastrófica tornaria as demais indistinguíveis entre si. O experimento totaliza 24 medições (12 camadas $\times$ 2 estratégias).

6.4.1 Perfil Obtido

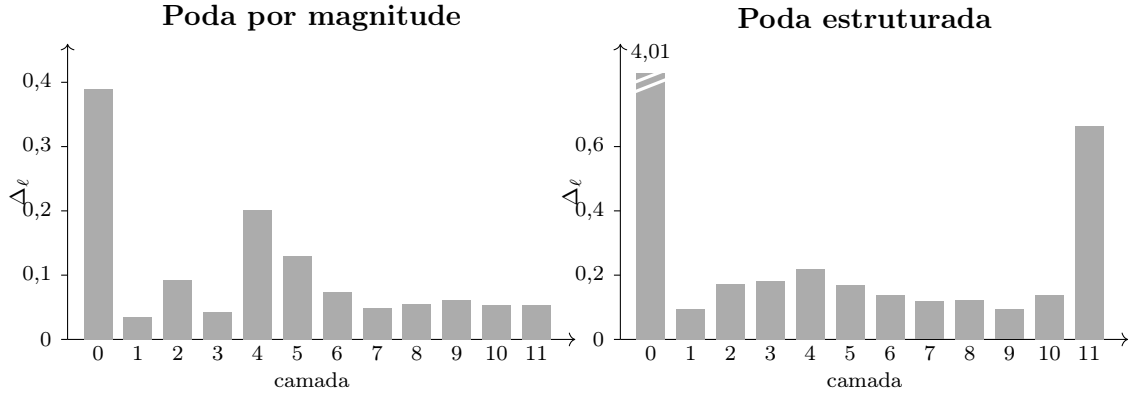
A Tabela 7 apresenta os resultados e a Figura 8 os representa graficamente.

Tabela 7 – Perfil de sensibilidade por camada do GPT-2 *small* no WikiText-2. Cada linha corresponde a podar exclusivamente a camada indicada, à taxa-sonda de 50 %, sem *fine-tuning*. Perplexity do modelo denso: 25,17.

Camada	Poda por magnitude		Poda estruturada	
	Perplexity	Δ_ℓ	Perplexity	Δ_ℓ
0	37,17	0,390	1386,43	4,009
1	26,06	0,035	27,66	0,094
2	27,62	0,093	29,91	0,173
3	26,28	0,043	30,23	0,183
4	30,81	0,202	31,31	0,218
5	28,66	0,130	29,80	0,169
6	27,09	0,074	28,90	0,138
7	26,44	0,049	28,31	0,118
8	26,59	0,055	28,45	0,122
9	26,76	0,061	27,63	0,093
10	26,55	0,054	28,89	0,138
11	26,55	0,054	48,87	0,663
Razão máx./mín.		11,3 $\times$	42,9 $\times$	

O primeiro resultado é que a hipótese se confirma de forma inequívoca: a sensibilidade está longe de ser uniforme. Sob poda por magnitude, a camada mais frágil degrada o modelo 11,3 vezes mais que a mais robusta; sob poda estruturada, 42,9 vezes mais. Tratar as 12 camadas como intercambiáveis — que é exatamente o que a alocação uniforme faz — ignora uma diferença de uma ordem de grandeza no custo de podar cada uma delas.

Figura 8 – Degradação Δ_ℓ causada pela poda isolada de cada camada, à taxa-sonda de 50 %. Note-se que as escalas verticais diferem entre os dois gráficos; na poda estruturada, a barra da camada 0 excede o eixo e está truncada.



Fonte: Elaborada pelo autor.

6.4.2 A Camada 0 como Gargalo

O segundo resultado é a magnitude do papel da primeira camada. Nas duas estratégias ela é a mais sensível, mas o efeito é qualitativamente distinto no caso estruturado: remover metade das cabeças de atenção e dos neurônios MLP **apenas do bloco 0** eleva a perplexity de 25,17 para 1386,43, ou seja, 55 vezes o valor do modelo denso. Para dimensionar esse número, a varredura estruturada uniforme a 50 % — que poda *todas* as 12 camadas — atinge perplexity 4573,00. Em escala logarítmica, a camada 0 sozinha responde por $4,01/5,20 \approx 77\%$ de toda a degradação produzida pela poda uniforme do modelo inteiro.

A interpretação é coerente com o papel funcional do primeiro bloco. É nele que as representações deixam de ser a soma direta de *embeddings* de token e de posição e passam a ser contextualizadas pela primeira vez; qualquer informação descartada ali não tem como ser recuperada pelas camadas subsequentes, que operam sobre um sinal já degradado. Trata-se, portanto, de um erro que se propaga por toda a pilha, ao contrário de um erro cometido nas camadas intermediárias.

Na poda estruturada observa-se ainda um segundo ponto crítico na outra extremidade: a camada 11, imediatamente anterior à cabeça de linguagem, apresenta $\Delta_{11} = 0,663$, quase cinco vezes a mediana das camadas intermediárias. O perfil estruturado tem, assim, formato aproximado de “U”: as bordas da pilha são críticas e o miolo é comparativamente barato. Esse padrão não se repete na poda por magnitude, cujo perfil decai após um pico inicial e mantém as últimas camadas entre as mais robustas.

6.4.3 Os Perfis Dependem da Estratégia

O terceiro resultado tem consequência metodológica direta: os dois perfis **não** são o mesmo. A correlação de postos de Spearman entre eles é $\rho = 0,50$ — há concordância, mas parcial. As duas estratégias coincidem em eleger a camada 0 como a mais crítica e em considerar as camadas 1 e 9 entre as mais robustas, e divergem em praticamente todo o resto: a camada 11 é a segunda mais sensível sob poda estruturada e apenas a oitava sob poda por magnitude; a camada 3 é a segunda mais robusta sob magnitude e a quarta mais sensível sob poda estruturada.

A implicação prática é que não existe um perfil de sensibilidade único do GPT-2. Sensibilidade é uma propriedade do par (camada, estratégia de poda), não da camada isoladamente, e o perfil precisa ser levantado separadamente para cada estratégia — como foi feito aqui — sob pena de se alocar o orçamento com um mapa que pertence a outro método. Esse achado também oferece uma explicação para a inversão observada nas varreduras ingênuas: como magnitude de peso e norma de estrutura respondem a aspectos diferentes da importância de uma camada, é esperado que o escopo global (que decide com base nessas grandezas) acerte em um caso e erre no outro.

6.4.4 Da Sensibilidade à Alocação

O perfil é um diagnóstico; para virar estratégia, precisa ser convertido em uma taxa de poda por camada. A regra adotada atribui a cada camada uma taxa proporcional à sua robustez — o inverso da degradação medida —, elevada a um expoente β e escalada por um fator único α tal que a média das taxas, ponderada pelo número de parâmetros de cada camada, iguale o orçamento global s pretendido:

$$r_\ell = \min\left(\alpha \cdot \Delta_\ell^{-\beta}, r_{\max}\right), \quad \frac{\sum_\ell w_\ell r_\ell}{\sum_\ell w_\ell} = s, \quad (6.2)$$

com w_ℓ o número de parâmetros podáveis da camada ℓ e $r_{\max} = 0,95$ um teto que impede que mesmo a camada mais robusta seja integralmente zerada. O fator α é obtido por bisseção, uma vez que a fração podada é monótona em α . O expoente β controla o quanto o perfil é levado a sério: com $\beta = 0$ a regra degenera na poda uniforme e com $\beta = 1$ usa-se o inverso puro da degradação; valores intermediários amortecem a razão entre camadas, o que importa porque essa razão é ilimitada — uma camada com Δ_ℓ muito próximo de zero receberia praticamente todo o orçamento.

A Tabela 8 mostra as taxas que a Equação 6.2 produziu para um orçamento global de 50% com $\beta = 1$. O contraste com a alocação uniforme é acentuado: sob poda estruturada, a camada 0 recebe 2% de poda e a camada 1 recebe 84%, enquanto a alocação uniforme aplicaria 50% a ambas.

Tabela 8 – Taxas de poda por camada efetivamente aplicadas pela Equação 6.2 para orçamento global de 50 % e $\beta = 1$ (coluna `taxas_por_camada` dos registros experimentais). A alocação uniforme corresponderia a 50 % em todas as camadas.

Camada	0	1	2	3	4	5	6	7	8	9	10	11
Magnitude (%)	8	94	35	75	16	25	44	67	59	53	61	61
Estruturada (%)	2	84	46	43	36	47	57	67	64	85	57	12

6.4.5 Limitações do Perfilamento

Três limitações do procedimento devem ser explicitadas, pois condicionam a leitura dos resultados da varredura otimizada.

A primeira é que a sonda foi aplicada a uma única taxa (50 %). O perfil mede, portanto, a sensibilidade das camadas naquele regime específico, e nada garante que a ordenação se mantenha em taxas muito menores ou muito maiores. A escolha por uma única sonda é uma concessão a custo: cada ponto do perfil exige uma avaliação completa de perplexity, e o levantamento com múltiplas sondas multiplicaria proporcionalmente o custo da fase.

A segunda, e mais relevante, é que a sonda mede o efeito de cada camada **isoladamente**, ignorando as interações entre elas. Os dados do próprio experimento mostram que essa aproximação é imperfeita: a soma dos Δ_ℓ individuais sob poda por magnitude é 1,238, ao passo que a poda uniforme simultânea de todas as camadas a 50 % produz degradação logarítmica de 2,934 — mais que o dobro. A degradação é fortemente superaditiva, isto é, os erros introduzidos em camadas distintas se compõem em vez de simplesmente se somarem. O perfil deve, por isso, ser lido como uma ordenação relativa de importância, e não como um modelo preditivo da degradação de uma configuração completa.

A terceira é que todas as medições são *one-shot*, sem *fine-tuning* de recuperação. Camadas que parecem frágeis nesse regime poderiam ser recuperáveis com retreinamento, o que eventualmente alteraria a alocação ótima. A opção pelo regime *one-shot* é deliberada e coerente com o objetivo do trabalho — isolar o efeito da poda do efeito do retreinamento —, mas restringe a generalidade das conclusões.

Nenhuma dessas limitações invalida o achado principal desta seção, que é robusto às três: sob qualquer leitura, as camadas do GPT-2 diferem entre si por mais de uma ordem de grandeza no custo de serem podadas, e essa diferença é informação que as alocações uniforme e global simplesmente não utilizam. Se essa informação se converte em ganho efetivo na fronteira entre compressão e qualidade é o que a varredura da estratégia otimizada, reportada na seção seguinte, se propõe a verificar.

6.5 Varredura da Alocação Guiada por Sensibilidade

Esta seção reporta a fase que responde à pergunta central do trabalho: a alocação derivada do perfil de sensibilidade melhora, de fato, o compromisso entre compressão e qualidade em relação às alocações ingênuas? As taxas por camada produzidas pela Equação 6.2 foram aplicadas nos mesmos nove níveis de orçamento das varreduras anteriores, para as duas estratégias, e avaliadas com o mesmo vetor de métricas. O experimento foi replicado com dois valores do expoente β — 1,0 e 0,5 —, o que fornece a ablação da função de alocação sobre um mesmo perfil.

6.5.1 Poda por Magnitude

A Tabela 9 compara as quatro alocações.

Tabela 9 – Perplexity da poda por magnitude sob as quatro alocações. Em negrito, o menor valor de cada linha.

Esparsidade	Alocação ingênua		Guiada por sensibilidade	
	Global	Uniforme	$\beta = 1,0$	$\beta = 0,5$
10 %	25,40	25,47	25,44	25,44
20 %	26,87	27,22	26,79	26,57
30 %	40,23	36,42	34,62	31,20
40 %	165,20	104,82	193,83	57,30
50 %	1096,50	473,28	1026,09	447,57
60 %	5121,25	2004,20	1524,55	2171,79
70 %	9371,16	6891,85	1853,30	2805,39
80 %	8847,27	10871,00	2747,56	3715,68
90 %	4612,69	13996,88	10398,10	7616,63

A alocação guiada vence em oito dos nove níveis, e as margens são substanciais na faixa de interesse. Em 30 % de esparsidade, a perplexity cai de 36,42 (melhor alocação ingênua) para 31,20 — uma redução de 14 %. Em 40 %, o ganho é de 104,82 para 57,30, ou seja, a degradação é praticamente **reduzida à metade** usando exatamente o mesmo orçamento de poda. Em 50 % a vantagem persiste, ainda que modesta (447,57 contra 473,28). Nos níveis altos, de 60 % a 80 %, a configuração com $\beta = 1$ produz perplexities entre 2,5 e 4 vezes menores que a melhor alocação ingênua — embora, nesse regime, todos os modelos já estejam degradados a ponto de a comparação ter interesse mais teórico que prático.

A única derrota ocorre em 90 %, onde o escopo global apresenta o menor valor. Como discutido na Seção 6.2, essa é a região em que a perplexity de todas as configu-

rações ultrapassa 4600 e a ordenação entre modelos igualmente inservíveis não carrega significado.

6.5.2 Poda Estruturada

Na poda estruturada, os ganhos são de outra ordem de grandeza, como mostra a Tabela 10.

Tabela 10 – Perplexity da poda estruturada sob as quatro alocações. Em negrito, o menor valor de cada linha.

Estruturas	Alocação ingênua		Guiada por sensibilidade	
	Global	Uniforme	$\beta = 1,0$	$\beta = 0,5$
10 %	356,20	481,13	124,29	215,83
20 %	2102,06	1484,89	371,07	717,19
30 %	2413,52	5467,56	724,53	923,65
40 %	2434,92	10486,71	1301,15	9489,87
50 %	2402,58	4573,00	2327,59	7922,81
60 %	2957,26	5799,76	5393,15	12322,05
70 %	4635,36	12249,58	10130,33	103463,60
80 %	7620,02	53313,38	8891,77	203995,84
90 %	5822,89	827316,19	5353,85	4830,24

Na faixa de 10 % a 50 % de estruturas removidas, a alocação guiada com $\beta = 1$ domina de forma inequívoca. Em 10 %, a perplexity cai de 356,20 para 124,29 — uma redução de 65 %. Em 20 %, de 1484,89 para 371,07, um fator de **quatro**. Em 30 %, de 2413,52 para 724,53, um fator de **3,3**. Considerando que a Seção 6.3 concluiu que não existe região gratuita para a poda estruturada *one-shot*, reduzir a perplexity a um quarto no mesmo orçamento é o resultado mais expressivo deste capítulo.

Uma ressalva quantitativa é necessária. Como a remoção de estruturas é discreta, as taxas fracionárias por camada não são atingíveis exatamente, e a alocação guiada acaba removendo uma fração de parâmetros ligeiramente menor que a ingênua no mesmo alvo: em 10 %, esparsidade real de 9,20 % contra 9,90 % do escopo global; em 30 %, 29,01 % contra 29,95 %. Parte da vantagem observada deve-se, portanto, a um corte marginalmente menor. A diferença é de menos de um ponto percentual de esparsidade, ao passo que a diferença de perplexity chega a um fator de quatro, de modo que a explicação não se sustenta como causa principal — mas o efeito existe e deve ser declarado.

A partir de 60 %, a ordenação se inverte e o escopo global volta a vencer. A causa é identificável nas taxas alocadas: a partir desse orçamento, um número crescente de camadas satura no teto $r_{\max} = 0,95$ — duas camadas em 60 %, quatro em 70 %, oito em 80 % e onze em 90 %. Saturadas as camadas, a alocação deixa de discriminar entre elas

e degenera em algo próximo da poda uniforme sobre o subconjunto saturado, ao mesmo tempo em que concentra uma pressão crescente sobre as poucas camadas ainda livres. A estratégia guiada, portanto, **só é informativa enquanto o orçamento couber dentro do teto por camada**; acima disso, ela perde sua vantagem estrutural. Esse é um limite intrínseco de qualquer alocação proporcional com teto, e não um defeito de implementação.

6.5.3 Ablação do Expoente β

A comparação entre $\beta = 1,0$ e $\beta = 0,5$ produz um resultado que não era antecipado: **o valor adequado de β depende da estratégia**, e depende dela de forma oposta nas duas.

Na poda estruturada, $\beta = 1$ domina em todos os níveis até 80 %, e o amortecimento é francamente prejudicial — em 70 % e 80 % a configuração com $\beta = 0,5$ produz perplexities de 103.463 e 203.996, uma ordem de grandeza pior que qualquer outra alocação testada. Na poda por magnitude ocorre o inverso na faixa útil: $\beta = 0,5$ é melhor de 20 % a 50 %, e só a partir de 60 % o expoente pleno passa à frente.

A explicação mais plausível liga-se à dispersão dos perfis. O perfil estruturado tem razão máximo/mínimo de $42,9\times$ e é dominado pela camada 0, cuja fragilidade é tão extrema (Seção 6.4.2) que qualquer amortecimento da alocação lhe devolve orçamento de poda com consequências catastróficas: em 70 %, $\beta = 0,5$ atribui 14,6 % de poda à camada 0, contra 3,0 % de $\beta = 1$. Quando o perfil contém um sinal dessa intensidade, convém segui-lo integralmente. Já o perfil de magnitude tem dispersão de $11,3\times$, distribuída entre várias camadas de sensibilidade intermediária; nele, a razão entre camadas é mais suscetível a ruído de medição, e amortecê-la funciona como regularização. O expoente β comporta-se, assim, como um parâmetro de confiança no perfil: quanto mais nítido o sinal, maior o β que compensa adotar.

6.5.4 Efeito sobre o Custo

Do ponto de vista da eficiência, a alocação guiada preserva integralmente o comportamento da estratégia de base, o que era esperado, uma vez que ela altera a distribuição do orçamento, não sua natureza. Na poda por magnitude, tamanho, FLOPs e tempo permanecem constantes, pelo mesmo motivo discutido na Seção 6.2. Na poda estruturada, o custo cai proporcionalmente: normalizando pelo *baseline* da própria sessão, a configuração com $\beta = 1$ registra tempo relativo de 0,659 e energia relativa de 0,674 em 50 %, e 0,419 e 0,384 em 90 % — valores equivalentes, dentro da variação entre sessões, aos 0,688 e 0,714 (50 %) e 0,445 e 0,413 (90 %) do escopo global.

O achado prático é, portanto, que **o ganho de qualidade da alocação guiada**

é gratuito em termos de custo: na faixa de 10 % a 50 % da poda estruturada, obtém-se uma perplexity de três a quatro vezes menor com essencialmente o mesmo tempo de inferência, o mesmo consumo de energia e o mesmo tamanho de modelo. É exatamente o deslocamento da fronteira entre compressão e qualidade que a segunda pergunta empírica do trabalho se propunha a investigar.

6.6 Síntese dos Resultados

Os experimentos permitem responder às duas perguntas que orientaram o trabalho.

À primeira — se a redução teórica de parâmetros se converte em redução real de consumo — a resposta é condicional à granularidade da poda. A poda não estruturada por magnitude reduz em 61 % os parâmetros não nulos sem alterar em nada o tempo de inferência, a energia, o tamanho em memória ou os FLOPs: na Tesla T4 utilizada, sem suporte de *hardware* para esparsidade, o ganho é puramente contábil. A poda estruturada, ao remover fisicamente cabeças de atenção e neurônios MLP, converte a mesma redução de 61 % em parâmetros em quedas de 56 % no tempo, 59 % na energia e 62 % nos FLOPs. Para o objetivo declarado deste trabalho — redução do consumo de energia —, apenas a poda estruturada é uma estratégia viável.

À segunda — se a alocação otimizada empurra a fronteira entre compressão e qualidade — a resposta é afirmativa, com escopo delimitado. As camadas do GPT-2 diferem entre si por mais de uma ordem de grandeza no custo de serem podadas, com a camada 0 respondendo isoladamente por cerca de três quartos da degradação da poda uniforme; essa informação, que nenhuma alocação ingênua utiliza, converte-se em ganho mensurável quando incorporada à distribuição do orçamento. Na poda estruturada — a estratégia que efetivamente reduz consumo — a alocação guiada reduz a perplexity por fatores de três a quatro na faixa de 10 % a 50 % de estruturas removidas, sem custo adicional de tempo, energia ou memória. O escopo da conclusão é delimitado por três fronteiras: o regime *one-shot*, a saturação da alocação em orçamentos acima de 60 %, e o fato de que o perfil de sensibilidade precisa ser levantado separadamente para cada estratégia de poda, por não ser transferível entre elas.

7 Considerações Finais

Referências

- BAHDANAU, D.; CHO, K.; BENGIO, Y. Neural machine translation by jointly learning to align and translate. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2015. Citado na página 11.
- BENGIO, Y.; SIMARD, P.; FRASCONI, P. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, v. 5, n. 2, p. 157–166, 1994. Citado na página 11.
- BLALOCK, D. et al. What is the state of neural network pruning? In: *Proceedings of Machine Learning and Systems (MLSys)*. [S.l.: s.n.], 2020. Citado 3 vezes nas páginas 17, 21 e 26.
- COURTY, B. et al. *CodeCarbon: estimate and track carbon emissions from your computing*. 2023. Disponível em: <<https://github.com/mlco2/codecarbon>>. Acesso em: 17 abr. 2026. Citado 2 vezes nas páginas 25 e 36.
- DENIL, M. et al. Predicting parameters in deep learning. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2013. Citado na página 16.
- FRANKLE, J.; CARBIN, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2019. Citado 2 vezes nas páginas 20 e 21.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge, MA: MIT Press, 2016. Disponível em: <<https://www.deeplearningbook.org/>>. Citado 5 vezes nas páginas 2, 4, 6, 8 e 24.
- HAN, S. et al. Learning both weights and connections for efficient neural networks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2015. Citado 10 vezes nas páginas 17, 19, 21, 23, 26, 27, 31, 34, 35 e 42.
- HENDRYCKS, D.; GIMPEL, K. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. Citado na página 13.
- HINTON, G.; VINYALS, O.; DEAN, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. Citado na página 16.
- HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997. Citado na página 11.
- JACOB, B. et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2018. Citado na página 16.
- KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. Imagenet classification with deep convolutional neural networks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2012. Citado na página 5.

- LI, H. et al. Pruning filters for efficient convnets. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2017. Citado 8 vezes nas páginas 21, 23, 26, 27, 32, 34, 43 e 44.
- LIU, Z. et al. Rethinking the value of network pruning. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2019. Citado na página 21.
- LUCCIONI, A. S.; VIGUIER, S.; LIGOZAT, A.-L. Estimating the carbon footprint of BLOOM, a 176B parameter language model. *arXiv preprint arXiv:2211.02001*, 2022. Citado na página 15.
- MICHEL, P.; LEVY, O.; NEUBIG, G. Are sixteen heads really better than one? In: *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2019. v. 32. Citado 2 vezes nas páginas 13 e 44.
- NAIR, V.; HINTON, G. E. Rectified linear units improve restricted Boltzmann machines. In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2010. Citado 2 vezes nas páginas 5 e 13.
- NIELSEN, M. A. *Neural Networks and Deep Learning*. [S.l.]: Determination Press, 2015. Disponível em: <<http://neuralnetworksanddeeplearning.com/>>. Citado na página 5.
- NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU Architecture*. 2020. White paper. Descreve o suporte a esparsidade estruturada 2:4 na arquitetura Ampere. Citado na página 19.
- PASZKE, A. et al. Pytorch: An imperative style, high-performance deep learning library. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2019. Citado 3 vezes nas páginas 22, 30 e 35.
- PATTERSON, D. et al. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021. Citado na página 15.
- RADFORD, A. et al. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019. Citado 2 vezes nas páginas 14 e 30.
- RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986. Citado na página 7.
- SEVILLA, J. et al. Compute trends across three eras of machine learning. *arXiv preprint arXiv:2202.05924*, 2022. Citado na página 16.
- STRUBELL, E.; GANESH, A.; MCCALLUM, A. Energy and policy considerations for deep learning in nlp. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2019. Citado na página 15.
- VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2017. p. 5998–6008. Citado 3 vezes nas páginas 10, 12 e 13.
- YANG, T.-J.; CHEN, Y.-H.; SZE, V. Designing energy-efficient convolutional neural networks using energy-aware pruning. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2017. Citado na página 21.

8 Apêndice

[TO DO]